VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB REPORT
on

Machine Learning (23CS6PCMAL)

Submitted by

Sharada Koundinya (1BM23CS310)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
Jan-2025 to May-2026

B.M.S. College of Engineering,

Bull Temple Road, Bangalore 560019

(Affiliated To Visvesvaraya Technological University, Belgaum)

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Lab work entitled "**Machine Learning (23CS6PCMAL)"** carried out by **Sharada Koundinya (1BM23CS310),** who is bona fide student of **B.M.S. College of Engineering.** It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum. The Lab report has been approved as it satisfies the academic requirements in respect of a Machine Learning (23CS6PCMAL) work prescribed for the said degree.

Ambuja K Assistant Professor Department of CSE, BMSCE	Dr. Kavitha Sooda Professor & HOD Department of CSE, BMSCE

Index

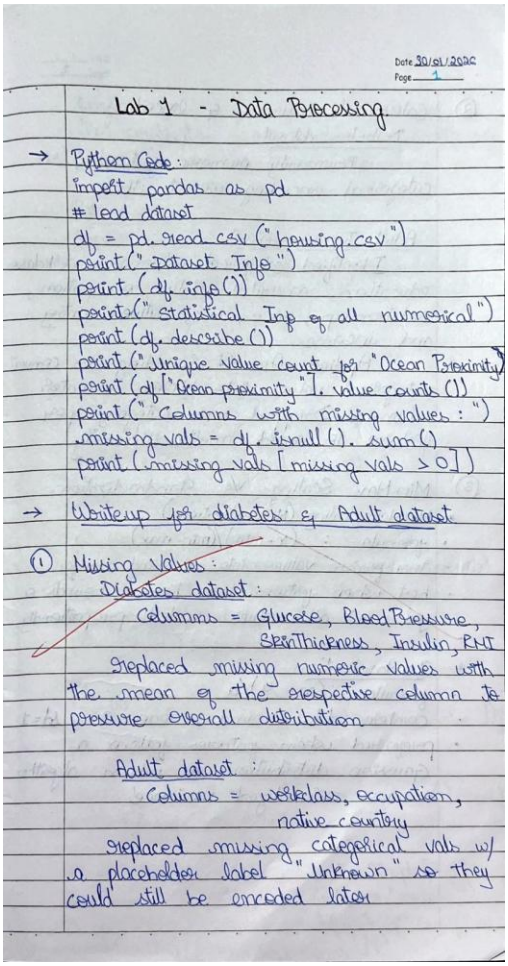
Sl. No.	Date	Experiment Title	Page No.
1		Write a python program to import and export data using Pandas library functions	4-9
2		Demonstrate various data pre-processing techniques for a given dataset	10-16
3		Implement Linear and Multi-Linear Regression algorithm using appropriate dataset	17-20
4		Build Logistic Regression Model for a given dataset	21-27
5		Use an appropriate data set for building the decision tree (ID3) and apply this knowledge to classify a new sample.	28-32
6		Build KNN Classification model for a given dataset.	33-35
7		Build Support vector machine model for a given dataset.	36-39
8		Implement Random forest ensemble method on a given dataset.	40-42
9		Implement Boosting ensemble method on a given dataset.	43-45
10		Build k-Means algorithm to cluster a set of data stored in a .CSV file.	46-48
11		Implement Dimensionality reduction using Principal Component Analysis (PCA) method.	49-51

Github Link:
https://github.com/sharadakoundinya/6Sem-ML-Lab

Program 1

Write a python program to import and export data using Pandas library functions

Screenshot



Date 30/01/2026
Page 1

Lab 1 - Data Processing

→ Python Code:

```
import pandas as pd
# load dataset
df = pd.read_csv("housing.csv")
print("Dataset Info")
print(df.info())
print("Statistical Info of all numerical")
print(df.describe())
print("Unique value count for 'Ocean Proximity')
print(df['Ocean proximity'].value_counts())
print("Columns with missing values:")
missing_vals = df.isnull().sum()
print(missing_vals[missing_vals > 0])
```

→ Writeup for diabetes & Adult dataset

① Missing Values:

Diabetes dataset:

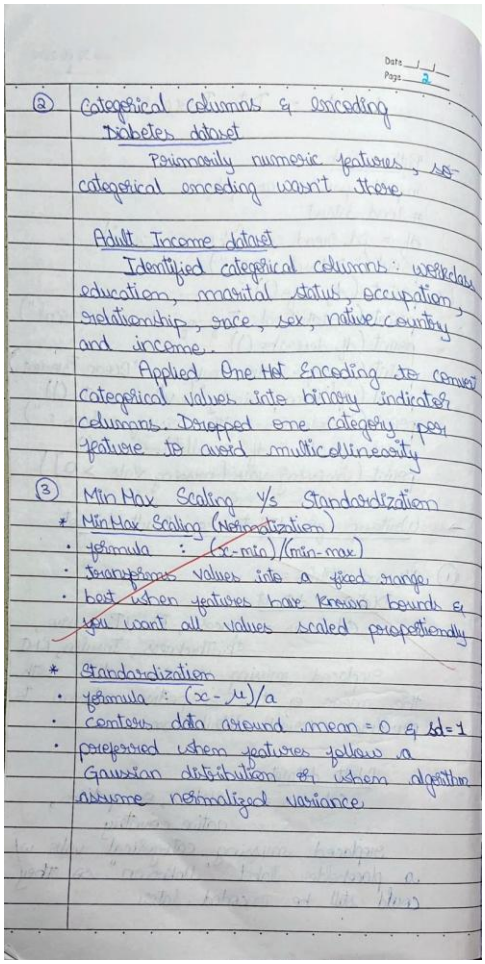
Columns = Glucose, BloodPressure, SkinThickness, Insulin, BMI

Replaced missing numeric values with the mean of the respective column to preserve overall distribution

Adult dataset:

Columns = workclass, occupation, native country

Replaced missing categorical vals w/ a placeholder label "Unknown" so they could still be encoded later



Date / /
Page 2

② Categorical columns & Encoding

Diabetes dataset

Primarily numeric features, so categorical encoding wasn't there

Adult Income dataset

Identified categorical columns: workclass, education, marital status, occupation, relationship, race, sex, native country and income.

Applied One Hot Encoding to convert categorical values into binary indicator columns. Dropped one category per feature to avoid multicollinearity

③ Min Max Scaling V/s Standardization

* Min Max Scaling (Normalization)
  - formula : (x-min)/(max-min)
  - transforms values into a fixed range
  - best when features have known bounds & you want all values scaled proportionally

* Standardization
  - formula : $(x-\mu)/\sigma$
  - centers data around mean = 0 & sd = 1
  - preferred when features follow a Gaussian distribution & when algorithm assume normalized variance

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OrdinalEncoder, OneHotEncoder
```

```
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from scipy import stats

def createdata():
  data = {
      'Age': np.random.randint(18, 70, size=20),
      'Salary': np.random.randint(30000, 120000, size=20),
      'Purchased': np.random.choice([0, 1], size=20),
      'Gender': np.random.choice(['Male', 'Female'], size=20),
      'City': np.random.choice(['New York', 'San Francisco', 'Los Angeles'], size=20)
  }

  df = pd.DataFrame(data)
  return df

df = createdata()
df.head(10)
```

	Age	Salary	Purchased	Gender	City
0	31	72779	1	Male	San Francisco
1	31	68565	1	Male	San Francisco
2	59	88399	1	Male	San Francisco
3	21	33182	0	Female	San Francisco
4	38	58152	1	Female	San Francisco
5	63	104883	0	Female	San Francisco
6	43	96859	0	Male	Los Angeles
7	20	109709	0	Female	San Francisco
8	62	82158	1	Male	San Francisco
9	28	99534	1	Male	San Francisco

```
df.shape
(20, 5)

df.loc[5, 'Age'] = np.nan
df.loc[10, 'Salary'] = np.nan
df.head(10)
```

	Age	Salary	Purchased	Gender	City
0	31.0	72779.0	1	Male	San Francisco
1	31.0	68565.0	1	Male	San Francisco
2	59.0	88399.0	1	Male	San Francisco
3	21.0	33182.0	0	Female	San Francisco
4	38.0	58152.0	1	Female	San Francisco
5	NaN	104883.0	0	Female	San Francisco
6	43.0	96859.0	0	Male	Los Angeles
7	20.0	109709.0	0	Female	San Francisco
8	62.0	82158.0	1	Male	San Francisco
9	28.0	99534.0	1	Male	San Francisco

```
print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20 entries, 0 to 19
Data columns (total 5 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   Age        19 non-null     float64
 1   Salary     19 non-null     float64
 2   Purchased  20 non-null     int64
 3   Gender     20 non-null     object
 4   City       20 non-null     object
dtypes: float64(2), int64(1), object(2)
memory usage: 932.0+ bytes
None
```

```
print(df.describe())
```

	Age	Salary	Purchased
count	19.000000	19.000000	20.000000
mean	39.631579	76143.631579	0.500000
std	14.407600	24171.475051	0.512989
min	18.000000	33182.000000	0.000000
25%	28.000000	59244.500000	0.000000
50%	38.000000	77331.000000	0.500000
75%	53.000000	95815.500000	1.000000
max	62.000000	112452.000000	1.000000

```
missing_values = df.isnull().sum()

print(missing_values[missing_values > 0])
```

```
Age       1
Salary    1
dtype: int64
```

```
imputer1 = SimpleImputer(strategy="median")
imputer2 = SimpleImputer(strategy="mean")

df_copy=df

imputer1.fit(df_copy[["Age"]])
imputer2.fit(df_copy[["Salary"]])

df_copy["Age"] = imputer1.transform(df[["Age"]])
df_copy["Salary"] = imputer2.transform(df[["Salary"]])

print(df_copy["Age"].isnull().sum())
print(df_copy["Salary"].isnull().sum())
```

```
0
0
```

```
ordinal_encoder = OrdinalEncoder(categories=[["Male", "Female"]])
df_copy["Gender_Encoded"] = ordinal_encoder.fit_transform(df_copy[["Gender"]])

onehot_encoder = OneHotEncoder()

encoded_data = onehot_encoder.fit_transform(df[["City"]])

encoded_array = encoded_data.toarray()

encoded_df = pd.DataFrame(encoded_array, columns=onehot_encoder.get_feature_names_out(["City"]))
df_encoded = pd.concat([df_copy, encoded_df], axis=1)

df_encoded.drop("Gender", axis=1, inplace=True)
df_encoded.drop("City", axis=1, inplace=True)

print(df_encoded. head())
```

```
...      Age    Salary  Purchased  Gender_Encoded  City_Los Angeles  City_New York  \
   0  52.0   44734.0          0             1.0               0.0            1.0
   1  49.0   67733.0          1             1.0               0.0            0.0
   2  62.0   57783.0          0             0.0               0.0            1.0
   3  45.0  104334.0          0             1.0               1.0            0.0
   4  47.0  100689.0          0             1.0               0.0            0.0

      City_San Francisco
   0                 0.0
   1                 1.0
   2                 0.0
   3                 0.0
   4                 1.0
```

```
normalizer = MinMaxScaler()
df_encoded[['Salary']] = normalizer.fit_transform(df_encoded[['Salary']])
df_encoded.head()
```

	Age	Salary	Purchased	Gender_Encoded	City_Los Angeles	City_New York	City_San Francisco
0	52.0	0.004300	0	1.0	0.0	1.0	0.0
1	49.0	0.320246	1	1.0	0.0	0.0	1.0
2	62.0	0.183559	0	0.0	0.0	1.0	0.0
3	45.0	0.823049	0	1.0	1.0	0.0	0.0
4	47.0	0.772976	0	1.0	0.0	0.0	1.0

```
scaler = StandardScaler()
df_encoded[['Age']] = scaler.fit_transform(df_encoded[['Age']])
df_encoded.head()
```

	Age	Salary	Purchased	Gender_Encoded	City_Los Angeles	City_New York	City_San Francisco
0	0.516451	0.004300	0	1.0	0.0	1.0	0.0
1	0.301263	0.320246	1	1.0	0.0	0.0	1.0
2	1.233744	0.183559	0	0.0	0.0	1.0	0.0
3	0.014346	0.823049	0	1.0	1.0	0.0	0.0
4	0.157804	0.772976	0	1.0	0.0	0.0	1.0

```
df_encoded_copy1=df_encoded
df_encoded_copy2=df_encoded
df_encoded_copy3=df_encoded

Q1 = df_encoded_copy1['Salary'].quantile(0.25)
Q3 = df_encoded_copy1['Salary'].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
df_encoded_copy1['Salary'] = np.where(df_encoded_copy1['Salary'] > upper_bound, upper_bound,
                    np.where(df_encoded_copy1['Salary'] < lower_bound, lower_bound,
df_encoded_copy1['Salary']))

print(df_encoded_copy1.head())
```

```
          Age    Salary  Purchased  Gender_Encoded  City_Los Angeles  \
0  0.516451  0.004300          0             1.0               0.0
1  0.301263  0.320246          1             1.0               0.0
2  1.233744  0.183559          0             0.0               0.0
3  0.014346  0.823049          0             1.0               1.0
4  0.157804  0.772976          0             1.0               0.0

   City_New York  City_San Francisco
0            1.0                0.0
1            0.0                1.0
2            1.0                0.0
3            0.0                0.0
4            0.0                1.0
```

```
df_encoded_copy2['Salary_zscore'] = stats.zscore(df_encoded_copy2['Salary'])
df_encoded_copy2['Salary'] = np.where(df_encoded_copy2['Salary_zscore'].abs() > 3, np.nan,
df_encoded_copy2['Salary'])  # Replace outliers with NaN
```

print(df_encoded_copy2.head())

```
...          Age    Salary  Purchased  Gender_Encoded  City_Los Angeles  \
    0  0.516451  0.004300          0             1.0               0.0
    1  0.301263  0.320246          1             1.0               0.0
    2  1.233744  0.183559          0             0.0               0.0
    3  0.014346  0.823049          0             1.0               1.0
    4  0.157804  0.772976          0             1.0               0.0

       City_New York  City_San Francisco  Salary_zscore
    0            1.0                 0.0      -1.449945
    1            0.0                 1.0      -0.447926
    2            1.0                 0.0      -0.881427
    3            0.0                 0.0       1.146703
    4            0.0                 1.0       0.987898
```

df_encoded_copy3['Salary_zscore'] = stats.zscore(df_encoded_copy3['Salary'])
median_salary = df_encoded_copy3['Salary'].median()
df_encoded_copy3['Salary'] = np.where(df_encoded_copy3['Salary_zscore'].abs() > 3, median_salary, df_encoded_copy3['Salary'])
print(df_encoded_copy3.head())

```
...          Age    Salary  Purchased  Gender_Encoded  City_Los Angeles  \
    0  0.516451  0.004300          0             1.0               0.0
    1  0.301263  0.320246          1             1.0               0.0
    2  1.233744  0.183559          0             0.0               0.0
    3  0.014346  0.823049          0             1.0               1.0
    4  0.157804  0.772976          0             1.0               0.0

       City_New York  City_San Francisco  Salary_zscore
    0            1.0                 0.0      -1.449945
    1            0.0                 1.0      -0.447926
    2            1.0                 0.0      -0.881427
    3            0.0                 0.0       1.146703
    4            0.0                 1.0       0.987898
```

Program 2

Demonstrate various data pre-processing techniques for a given dataset

Screenshot

Date 06/02/2026
Page 3

Lab 2
Demonstrate various data pre processing techniques for a given dataset

Python codes

```
import pandas as pd
url = "https://raw.githubusercontent.com/ageron/handson-ml/master/datasets/housing/housing.csv"

housing.head(10)
housing.dtypes
housing.describe

import matplotlib.pyplot as plt
import seaborn as sns

plt.figure()
housing['median_house_value'].hist(bins=50)
plt.xlabel("Median Housing Value")
plt.ylabel("Frequency")
plt.show()

plt.figure()
sns.boxplot(x=housing['median_house_value'])
plt.show()

missing_percentage = housing.isnull().mean()
missing_percentage

from sklearn.impute import SimpleImputer
imputer = SimpleImputer(strategy="median")
housing['total_bedrooms'] = imputer.fit_transform(housing[['total_bedrooms']])
```

Date / /
Page

```
housing_encoded = pd.get_dummies(
    housing, columns=['ocean_proximity'],
    drop_first=True)

from sklearn.preprocessing import MinMaxScaler
minmax_scaler = MinMaxScaler(feature_range=(0,1))
numeric_features = housing_encoded.select_dtypes
    (include=['float64','int64'])
housing_encoded[numeric_features] = minmax_scaler

from sklearn.model_selection import StratifiedS
import numpy as np

housing['income_cat'] = pd.cut(
    housing['median_income'],
    bins=[0, 1.5, 3.0, 4.5, 6.0, np.inf],
    labels=[1,2,3,4,5])

housing_encoded['income_cat'] = housing['income_cat']

split = StratifiedShuffleSplit(n_splits=1, test_size=0.2
    random_state=42)

for train_idx, test_idx in split.split
(housing_encoded, housing_encoded['income_cat'])
    start_train = housing_encoded.iloc[train_idx].drop('income_cat', axis=1, inplace=1)
    start_test = housing_encoded.iloc[test_idx]

corr_matrix = start_train.corr()

plt.figure(figsize=(12,8))
sns.heatmap(corr_matrix, cmap="coolwarm")
plt.show()
```

Date / /
Page 5

```
plt.figure()
plt.scatter(
    housing['median_income'],
    housing['median_house_value'],
    alpha=0.1)

plt.xlabel("Median Income")
plt.ylabel("Median House Value")
plt.title("Median vs Median House Value")
plt.show
```

Trend observed
As median income ↑s, house value ↑s

Linear or Nonlinear
mostly linear, but capped at upper limit

Outliers/Clusters
clear horizontal cap
dense clusters at lower income ranges

```
from sklearn.model_selection import train_test_split

train_set, test_set, = train_test_split(
    housing_encoded, test_size=0.2,
    random_state=42)

corr_target = corr_matrix['median_housing_value'].sort_values(ascending=False)
corr_target
```

Date / /
Page

```
from sklearn.linear_model import LinearRegression

X_train = train_set.drop('median_house', axis=1)
Y_train = train_set['median_house_value']

X_test = test_set.drop('median_house', axis=1)
Y_test = test_set['median_house_value']

lin_reg = LinearRegression()
lin_reg.fit(X_train, Y_train)

Y_pred=(lin_reg.predict(X_test))

rmse = np.sqrt(mean_squared_error(Y_test, Y_pred))
r2 = r2_score(Y_test, Y_pred)
mae = mean_absolute_error(Y_test, Y_pred)

rmse, mae, r2
```

rmse - 0.605787
mae - 0.439867
r2 - 0.6256362

Code:

```
import pandas as pd
url = "https://raw.githubusercontent.com/ageron/handson-ml/master/datasets/housing/housing.csv"
housing = pd.read_csv(url)

housing.head(10)
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	population	households	median_income	median_house_value	ocean_proximity
0	-122.23	37.88	41.0	880.0	129.0	322.0	126.0	8.3252	452600.0	NEAR BAY
1	-122.22	37.86	21.0	7099.0	1106.0	2401.0	1138.0	8.3014	358500.0	NEAR BAY
2	-122.24	37.85	52.0	1467.0	190.0	496.0	177.0	7.2574	352100.0	NEAR BAY
3	-122.25	37.85	52.0	1274.0	235.0	558.0	219.0	5.6431	341300.0	NEAR BAY
4	-122.25	37.85	52.0	1627.0	280.0	565.0	259.0	3.8462	342200.0	NEAR BAY
5	-122.25	37.85	52.0	919.0	213.0	413.0	193.0	4.0368	269700.0	NEAR BAY
6	-122.25	37.84	52.0	2535.0	489.0	1094.0	514.0	3.6591	299200.0	NEAR BAY
7	-122.25	37.84	52.0	3104.0	687.0	1157.0	647.0	3.1200	241400.0	NEAR BAY
8	-122.26	37.84	42.0	2555.0	665.0	1206.0	595.0	2.0804	226700.0	NEAR BAY
9	-122.25	37.84	52.0	3549.0	707.0	1551.0	714.0	3.6912	261100.0	NEAR BAY

```
housing.dtypes
```

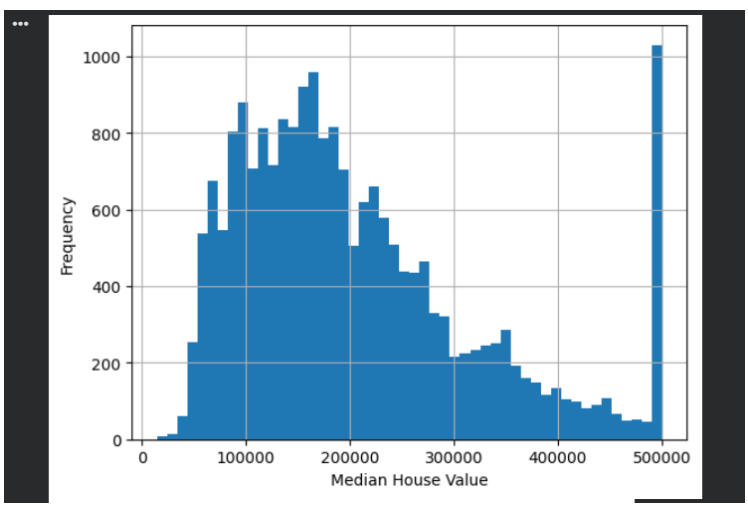
	0
longitude	float64
latitude	float64
housing_median_age	float64
total_rooms	float64
total_bedrooms	float64
population	float64
households	float64
median_income	float64
median_house_value	float64
ocean_proximity	object

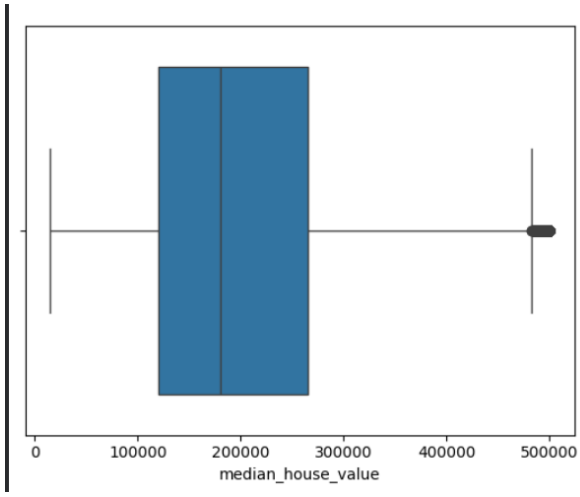
dtype: object

housing.describe()

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	population	households	median_income	median_house_value
count	20640.000000	20640.000000	20640.000000	20640.000000	20433.000000	20640.000000	20640.000000	20640.000000	20640.000000
mean	-119.569704	35.631861	28.639486	2635.763081	537.870553	1425.476744	499.539680	3.870671	206855.816909
std	2.003532	2.135952	12.585558	2181.615252	421.385070	1132.462122	382.329753	1.899822	115395.615874
min	-124.350000	32.540000	1.000000	2.000000	1.000000	3.000000	1.000000	0.499900	14999.000000
25%	-121.800000	33.930000	18.000000	1447.750000	296.000000	787.000000	280.000000	2.563400	119600.000000
50%	-118.490000	34.260000	29.000000	2127.000000	435.000000	1166.000000	409.000000	3.534800	179700.000000
75%	-118.010000	37.710000	37.000000	3148.000000	647.000000	1725.000000	605.000000	4.743250	264725.000000
max	-114.310000	41.950000	52.000000	39320.000000	6445.000000	35682.000000	6082.000000	15.000100	500001.000000

```
import matplotlib.pyplot as plt
import seaborn as sns

# Histogram
plt.figure()
housing['median_house_value'].hist(bins=50)
plt.xlabel("Median House Value")
plt.ylabel("Frequency")
plt.show()

# Box Plot
plt.figure()
sns.boxplot(x=housing['median_house_value'])
plt.show()
```





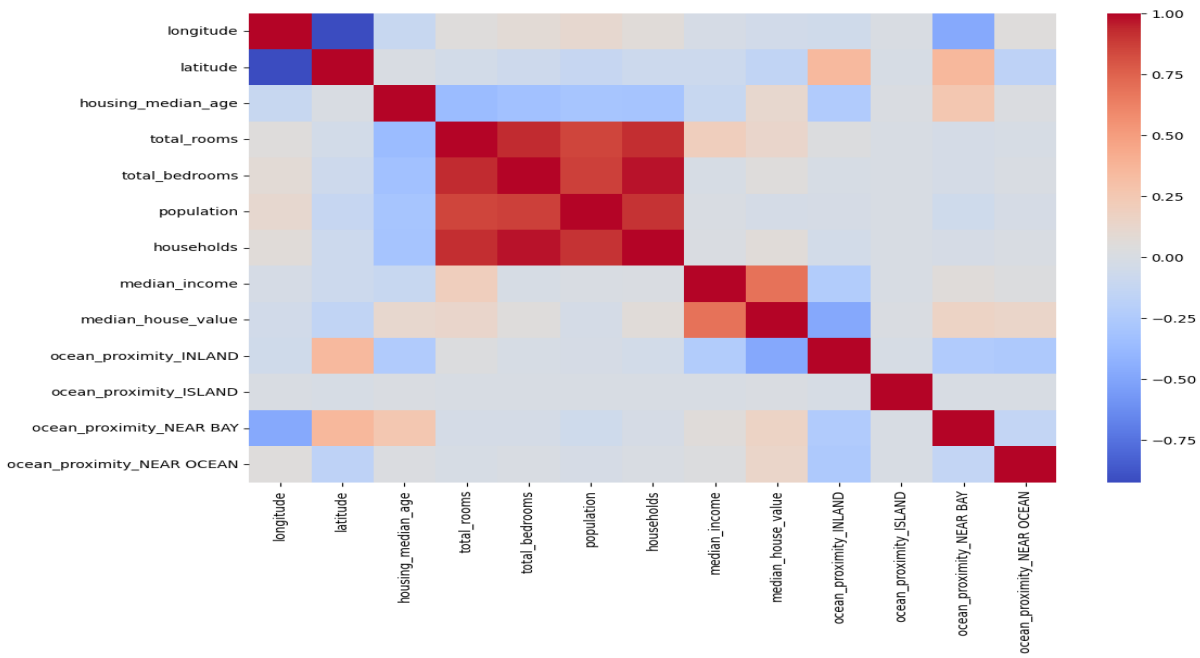
```
missing_percentage = housing.isnull().mean() * 100
missing_percentage
```

	0
longitude	0.000000
latitude	0.000000
housing_median_age	0.000000
total_rooms	0.000000
total_bedrooms	1.002907
population	0.000000
households	0.000000
median_income	0.000000
median_house_value	0.000000
ocean_proximity	0.000000

dtype: float64

```
from sklearn.impute import SimpleImputer

imputer = SimpleImputer(strategy="median")
housing['total_bedrooms'] = imputer.fit_transform(
    housing[['total_bedrooms']]
)

housing_encoded = pd.get_dummies(
    housing,
    columns=['ocean_proximity'],
    drop_first=True
)

from sklearn.preprocessing import MinMaxScaler

minmax_scaler = MinMaxScaler(feature_range=(0, 1))

numeric_features = housing_encoded.select_dtypes(include=['float64', 'int64']).columns

housing_encoded[numeric_features] = minmax_scaler.fit_transform(
    housing_encoded[numeric_features]
)

from sklearn.model_selection import StratifiedShuffleSplit
import numpy as np

# Create 'income_cat' on the original housing DataFrame first
```

```
housing['income_cat'] = pd.cut(
    housing['median_income'],
    bins=[0, 1.5, 3.0, 4.5, 6.0, np.inf],
    labels=[1, 2, 3, 4, 5]
)

# Add 'income_cat' to the housing_encoded DataFrame
housing_encoded['income_cat'] = housing['income_cat']

split = StratifiedShuffleSplit(n_splits=1, test_size=0.2, random_state=42)

for train_idx, test_idx in split.split(housing_encoded, housing_encoded['income_cat']):
    strat_train = housing_encoded.loc[train_idx]
    strat_test = housing_encoded.loc[test_idx]

strat_train.drop('income_cat', axis=1, inplace=True)
strat_test.drop('income_cat', axis=1, inplace=True)
corr_matrix = strat_train.corr()
plt.figure(figsize=(12,8))
sns.heatmap(corr_matrix, cmap="coolwarm")
plt.show()
```




```
plt.figure()
plt.scatter(
    housing['median_income'],
    housing['median_house_value'],
    alpha=0.1
)
```

```
plt.xlabel("Median Income")
plt.ylabel("Median House Value")
plt.title("Median Income vs Median House Value (Original Data)")
plt.show()
```

```
from sklearn.model_selection import train_test_split

train_set, test_set = train_test_split(
    housing_encoded,
    test_size=0.2,
    random_state=42
)
corr_target = corr_matrix['median_house_value'].sort_values(ascending=False)
corr_target
```

	median_house_value
median_house_value	1.000000
median_income	0.687151
ocean_proximity_NEAR BAY	0.158691
ocean_proximity_NEAR OCEAN	0.137332
total_rooms	0.135140
housing_median_age	0.114146
households	0.064590
total_bedrooms	0.047732
ocean_proximity_ISLAND	0.013708
population	-0.026882
longitude	-0.047466
latitude	-0.142673
ocean_proximity_INLAND	-0.482853

dtype: float64

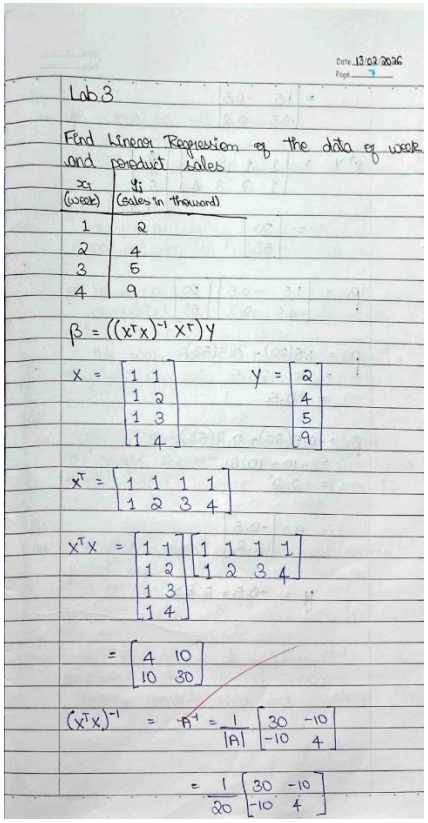
```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
train_set[numeric_features] = scaler.fit_transform(train_set[numeric_features])
test_set[numeric_features] = scaler.transform(test_set[numeric_features])

from sklearn.linear_model import LinearRegression

X_train = train_set.drop('median_house_value', axis=1)
y_train = train_set['median_house_value']

X_test = test_set.drop('median_house_value', axis=1)
y_test = test_set['median_house_value']

lin_reg = LinearRegression()
lin_reg.fit(X_train, y_train)
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
import numpy as np

y_pred = lin_reg.predict(X_test)

rmse = np.sqrt(mean_squared_error(y_test, y_pred))
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

rmse, mae, r2
```

```
(np.float64(0.605787984146388), 0.43986704551058153, 0.6256362335780752)
```

Program 3

Implement Linear and Multi-Linear Regression algorithm using appropriate dataset

Screenshot



Date 13/02/2026
Page 7

Lab 3

Find Linear Regression of the data of week and predict sales

x_i (week)	y_i (sales in thousand)
1	2
2	4
3	5
4	9

$\beta = ((X^T X)^{-1} X^T) Y$

$X = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \\ 1 & 4 \end{bmatrix} \quad Y = \begin{bmatrix} 2 \\ 4 \\ 5 \\ 9 \end{bmatrix}$

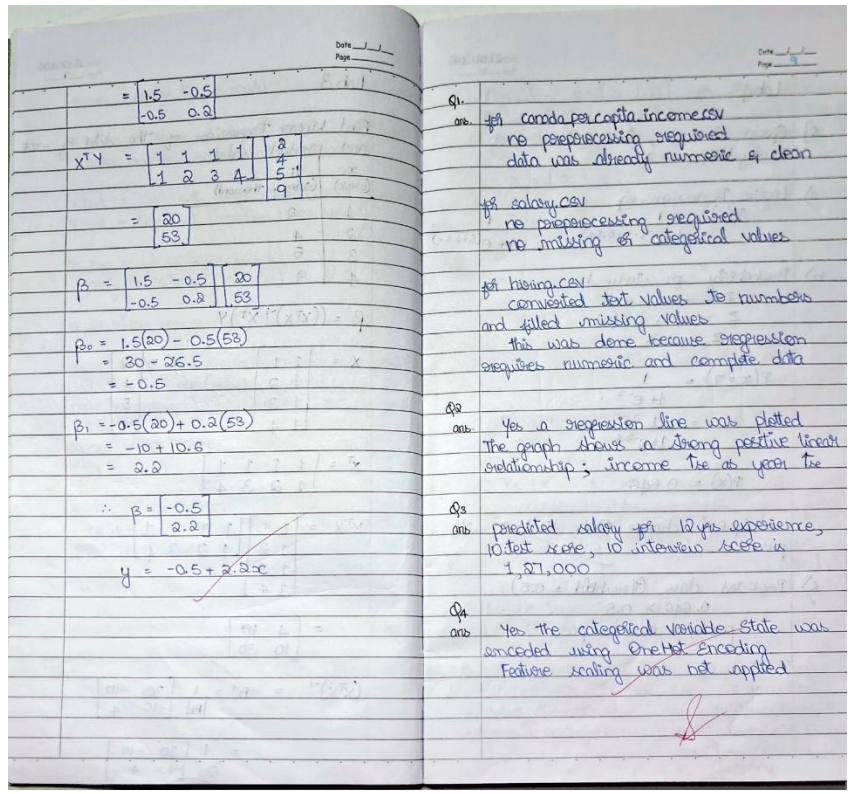
$X^T = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 \end{bmatrix}$

$X^T X = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \\ 1 & 4 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 \end{bmatrix}$

$= \begin{bmatrix} 4 & 10 \\ 10 & 30 \end{bmatrix}$

$(X^T X)^{-1} = A^{-1} = \frac{1}{|A|} \begin{bmatrix} 30 & -10 \\ -10 & 4 \end{bmatrix}$

$= \frac{1}{20} \begin{bmatrix} 30 & -10 \\ -10 & 4 \end{bmatrix}$



$= \begin{bmatrix} 1.5 & -0.5 \\ -0.5 & 0.2 \end{bmatrix}$

$X^T Y = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 \end{bmatrix} \begin{bmatrix} 2 \\ 4 \\ 5 \\ 9 \end{bmatrix}$

$= \begin{bmatrix} 20 \\ 53 \end{bmatrix}$

$\beta = \begin{bmatrix} 1.5 & -0.5 \\ -0.5 & 0.2 \end{bmatrix} \begin{bmatrix} 20 \\ 53 \end{bmatrix}$

$\beta_0 = 1.5(20) - 0.5(53)$
$= 30 - 26.5$
$= -0.5$

$\beta_1 = -0.5(20) + 0.2(53)$
$= -10 + 10.6$
$= 2.2$

$\therefore \beta = \begin{bmatrix} -0.5 \\ 2.2 \end{bmatrix}$

$y = -0.5 + 2.2x$

Date / /
Page 9

Q1. ans. for canada_per_capita_income.csv
no preprocessing required
data was already numeric & clean

for salary.csv
no preprocessing required
no missing or categorical values

for hiring.csv
converted text values to numbers and filled missing values
this was done because regression requires numeric and complete data

Q2 ans. Yes a regression line was plotted
The graph shows a strong positive linear relationship ; income rise as years rise

Q3 ans. predicted salary for 12 yrs experience, 10 test score, 10 interview score is 1,27,000

Q4 ans. Yes the categorical variable State was encoded using OneHot Encoding
Feature scaling was not applied

Code:

```
import numpy as np

X = np.array([1,2,3,4]).reshape(-1,1)
Y = np.array([2,4,5,9]).reshape(-1,1)

X_design = np.hstack([np.ones((X.shape[0],1)), X])

beta = np.linalg.inv(X_design.T @ X_design) @ X_design.T @ Y
print("Intercept:", beta[0][0])
print("Slope:", beta[1][0])
```

Intercept: -0.5
Slope: 2.2000000000000006

```
import pandas as pd
import numpy as np

df = pd.read_csv('canada_per_capita_income.csv')
```

```
X = df[['year']].values
Y = df.iloc[:,1].values.reshape(-1,1)

X_design = np.hstack([np.ones((X.shape[0],1)), X])

beta = np.linalg.inv(X_design.T @ X_design) @ X_design.T @ Y

pred = np.array([1,2020]) @ beta
print("Predicted income:", pred[0])
```

Predicted income: 41288.694094381295

```
df = pd.read_csv('salary.csv')
df = df.dropna()

X = df.iloc[:,0].values.reshape(-1,1)
Y = df.iloc[:,1].values.reshape(-1,1)

X_design = np.hstack([np.ones((X.shape[0],1)), X])

beta = np.linalg.inv(X_design.T @ X_design) @ X_design.T @ Y

pred = np.array([1,12]) @ beta
print("Predicted Salary:", pred[0])
```

Predicted Salary: 139049.6749539778

```
from sklearn.linear_model import LinearRegression

df = pd.read_csv('hiring.csv')

word_to_num = {
    'zero':0,
    'one':1,
    'two':2,
    'three':3,
    'four':4,
    'five':5,
    'six':6,
    'seven':7,
    'eight':8,
    'nine':9,
    'ten':10,
    'eleven':11,
    'twelve':12
}
```

```
df['experience'] = df['experience'].map(word_to_num)

df['experience'] = df['experience'].fillna(0)

df['test_score(out of 10)'] = df['test_score(out of 10)'].fillna(
    df['test_score(out of 10)'].median()
)

X = df[['experience',
        'test_score(out of 10)',
        'interview_score(out of 10)']]

y = df['salary($)']

model = LinearRegression()
model.fit(X, y)

pred1 = model.predict([[2,9,6]])
pred2 = model.predict([[12,10,10]])

print("Salary Prediction 1:", pred1[0])
print("Salary Prediction 2:", pred2[0])
```

Salary Prediction 1: 53205.96797671033
Salary Prediction 2: 92002.18340611353

```
df = pd.read_csv('1000_Companies.csv')

df = pd.get_dummies(df, columns=['State'], drop_first=True)

X = df.drop('Profit', axis=1)
y = df['Profit']

model = LinearRegression()
model.fit(X, y)

input_data = [[91694.48, 515841.3, 11931.24, 0, 1]]
pred_profit = model.predict(input_data)

print("Predicted Profit:", pred_profit[0])
```
Predicted Profit: 511115.0737557667

```
print(X.columns)
```
Index(['R&D Spend', 'Administration', 'Marketing Spend', 'State_Florida',
'State_New York'],
dtype='object')

```
X = np.array([1,2,3,4]).reshape(-1,1)
```

```
y = np.array([2,4,5,9])

model = LinearRegression()
model.fit(X, y)

print("Intercept:", model.intercept_)
print("Slope:", model.coef_[0])

print("Prediction for week 5:", model.predict([[5]]))
```

Intercept: -0.4999999999999991
Slope: 2.1999999999999997
Prediction for week 5: [10.5]

```
X = np.array([1,2,3,4]).reshape(-1,1)
Y = np.array([2,4,5,9]).reshape(-1,1)

X_design = np.hstack([np.ones((X.shape[0],1)), X])

beta = np.linalg.inv(X_design.T @ X_design) @ X_design.T @ Y

print("Intercept:", beta[0][0])
print("Slope:", beta[1][0])
```

Intercept: -0.5
Slope: 2.2000000000000006

```
df = pd.read_csv('hiring.csv')

mapping = {'zero':0,'one':1,'two':2,'three':3,'four':4,
        'five':5,'six':6,'seven':7,'eight':8,'nine':9,'ten':10}
df['experience'] = df['experience'].map(mapping).fillna(0)
df['test_score(out of 10)'] = df['test_score(out of 10)'].fillna(
                        df['test_score(out of 10)'].median())

X = df[['experience','test_score(out of 10)','interview_score(out of 10)']]
y = df['salary($)']

model = LinearRegression()
model.fit(X,y)

print(model.predict([[12,10,10]]))
```

[79138.40026552]

Program 4

Build Logistic Regression Model for a given dataset

Screenshot

Lab4

a] Given $a_0 = -5$, $a_1 = 0.8$
Study hours = x

a) Logistic Regression eq
$P(x) = \frac{1}{1+e^{-(-5+0.8x)}} = \frac{1}{1+e^{(5+0.8x)}}$

b) Probability for study hours = 7
$z = -5 + 0.8x$
$z = -5 + 0.8(7)$
$z = 0.6$

$P(x=7) = \frac{1}{1+e^{-z}}$
$= \frac{1}{1+e^{-0.6}}$

$P(x) = 0.646$

∴ 64.6% chance that student will pass

c) Predicted class (threshold = 0.5)
$0.646 > 0.5$
predicted class = pass

a] Consider $z = [2,1,0]$ for three classes. Apply Softmax function to find the probability values of three classes

$z = [2,1,0]$

$P_i = \frac{e^{z_i}}{\Sigma e^{z_j}}$

$\Sigma e^{z_i} = e^2 + e^1 + e^0$
$= 7.389 + 2.718 + 1$
sum $= 11.107$

$P_1 = \frac{7.389}{11.107} = 0.665$

$P_2 = \frac{2.718}{11.107} = 0.245$

$P_3 = \frac{1}{11.107} = 0.090$

Softmax $([2,1,0]) = [0.665, 0.245, 0.090]$

Logistic Regression models
1. For dataset "HR_comma_sep.csv"

i) Variables affecting employee retention
Direct impact on them are:
- satisfaction level - low satisfaction, employee leaves
- avg_monthly_hours - very high working hrs increases chances of leaving

- time_spend_company - employee staying many years usually growth tend to leave
- promotion_last_5yrs - ussual promotion are more likely to leave
- salary - employee w/ low salary leave more frequently

ii) accuracy of logistic regression
The model accuracy was approx 75-80%
This is good accuracy because employee behaviour is difficult to predict perfectly.

2. Zoo Dataset:
i) Yes, The animal_name column was removed because its a text field & doesn't help in predicting class type.

ii) There were no missing or inconsistent values in the dataset so no special handling was required

iii) The confusion matrix shows how many predictions were correct and incorrect for each class. It helps evaluate the performance of the classification model

iv) A few classes with similar characteristics were sometimes misclassified, because some animals share similar features like legs, fins, eggs

Code:

```
import pandas as pd
df = pd.read_csv("HR_comma_sep.csv")

import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report

print(df.head())
print(df.isnull().sum())
```

```
      satisfaction_level  last_evaluation  number_project  average_montly_hours  \
...  0                0.38             0.53               2                   157
     1                0.80             0.86               5                   262
     2                0.11             0.88               7                   272
     3                0.72             0.87               5                   223
     4                0.37             0.52               2                   159

      time_spend_company  Work_accident  left  promotion_last_5years Department  \
     0                   3              0     1                      0      sales
     1                   6              0     1                      0      sales
     2                   4              0     1                      0      sales
     3                   5              0     1                      0      sales
     4                   3              0     1                      0      sales

      salary
     0    low
     1 medium
     2 medium
     3    low
     4    low
     <class 'pandas.core.frame.DataFrame'>
     RangeIndex: 14999 entries, 0 to 14998
```

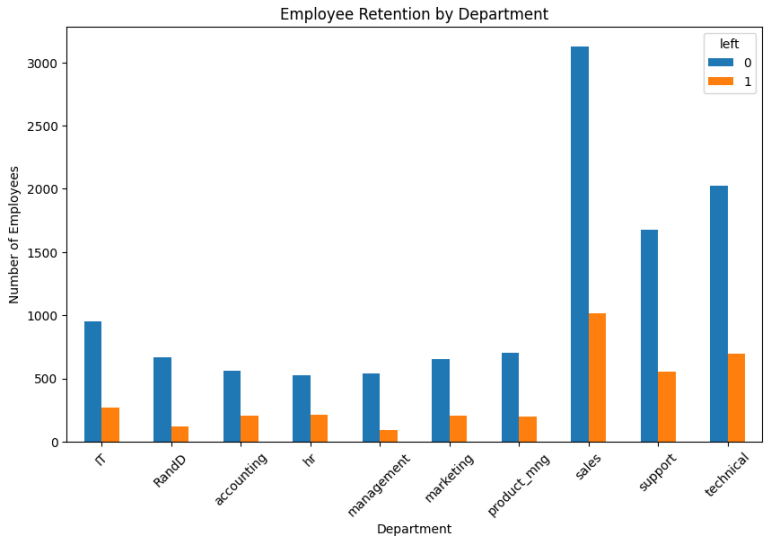
```
corr = df.corr(numeric_only=True)
print(corr['left'].sort_values(ascending=False))
```

```
...  left                     1.000000
     time_spend_company       0.144822
     average_montly_hours     0.071287
     number_project           0.023787
     last_evaluation          0.006567
     promotion_last_5years   -0.061788
     Work_accident           -0.154622
     satisfaction_level      -0.388375
     Name: left, dtype: float64
```

```
salary_retention = pd.crosstab(df['salary'], df['left'])

salary_retention.plot(kind='bar')
plt.title("Employee Retention Based on Salary")
plt.xlabel("Salary Level")
plt.ylabel("Number of Employees")
```

```
plt.show()
```

```
dept_retention = pd.crosstab(df['Department'], df['left'])

dept_retention.plot(kind='bar', figsize=(10,6))
plt.title("Employee Retention by Department")
plt.xlabel("Department")
plt.ylabel("Number of Employees")
plt.xticks(rotation=45)
plt.show()
```




```
X = df[['satisfaction_level',
        'average_montly_hours',
        'promotion_last_5years',
        'time_spend_company']]
```

```
salary_dummies = pd.get_dummies(df['salary'], drop_first=True)
X = pd.concat([X, salary_dummies], axis=1)
y = df['left']


X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print("Model Accuracy:", accuracy)
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

```
Model Accuracy: 0.7566666666666667

Classification Report:
              precision    recall  f1-score   support

           0       0.80      0.92      0.85      2294
           1       0.47      0.24      0.32       706

    accuracy                           0.76      3000
   macro avg       0.63      0.58      0.58      3000
weighted avg       0.72      0.76      0.73      3000
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

zoo = pd.read_csv('zoo-data.csv')
print(zoo.head())
print(zoo.dtypes)
```

```
...    animal_name  hair  feathers  eggs  milk  airborne  aquatic  predator  \
0    aardvark     1        0     0     1        0        0         1
1    antelope     1        0     0     1        0        0         0
2        bass     0        0     1     0        0        1         1
3        bear     1        0     0     1        0        0         1
4        boar     1        0     0     1        0        0         1

   toothed  backbone  breathes  venomous  fins  legs  tail  domestic  catsize  \
0        1         1         1         0     0     4     0         0        1
1        1         1         1         0     0     4     1         0        1
2        1         1         0         0     1     0     1         0        0
3        1         1         1         0     0     4     0         0        1
4        1         1         1         0     0     4     1         0        1

   class_type
0           1
1           1
2           4
3           1
4           1
animal_name    object
hair           int64
feathers       int64
eggs           int64
milk           int64
airborne       int64
aquatic        int64
predator       int64
toothed        int64
backbone       int64
breathes       int64
venomous       int64
fins           int64
legs           int64
tail           int64
domestic       int64
catsize        int64
class_type     int64
dtype: object
```

zoo.drop('animal_name', axis=1, inplace=True)
print(zoo.dtypes)

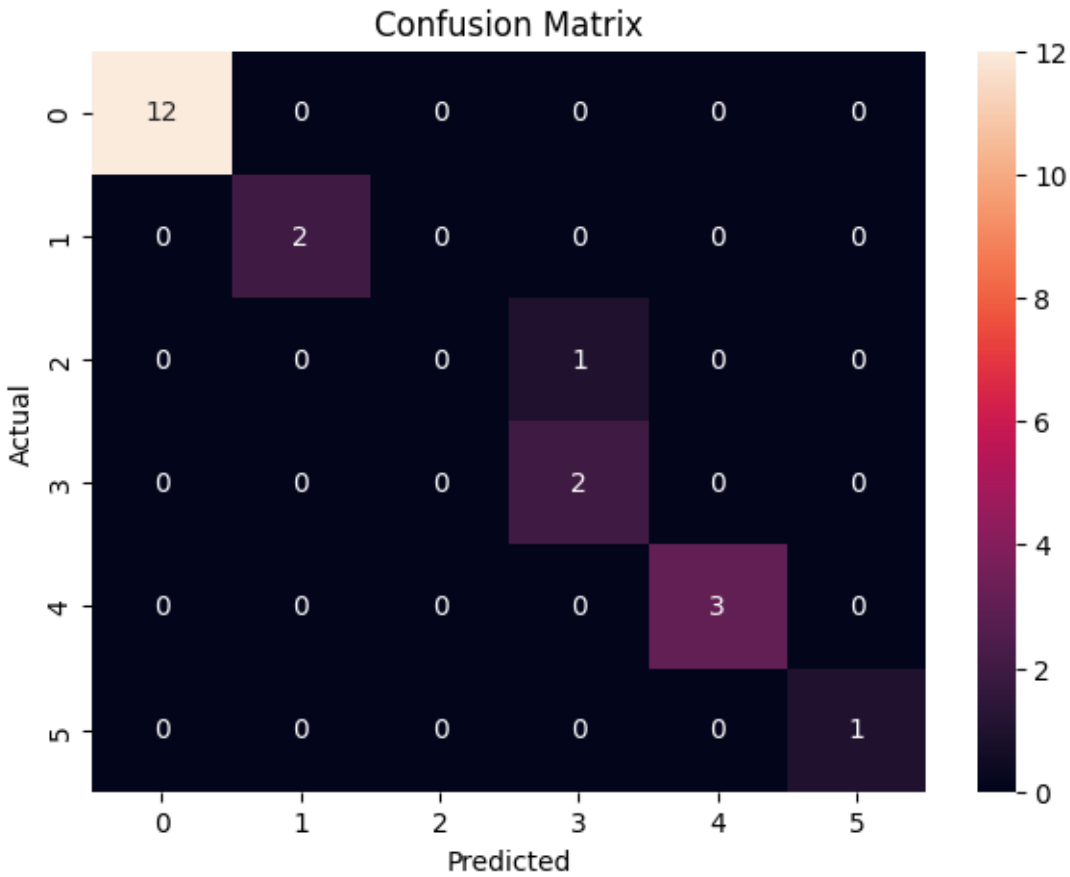
```
hair           int64
feathers       int64
eggs           int64
milk           int64
airborne       int64
aquatic        int64
predator       int64
toothed        int64
backbone       int64
breathes       int64
venomous       int64
fins           int64
legs           int64
tail           int64
domestic       int64
catsize        int64
class_type     int64
dtype: object
```

X = zoo.drop('class_type', axis=1)

```
y = zoo['class_type']

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

from sklearn.linear_model import LogisticRegression

model = LogisticRegression(max_iter=5000)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

from sklearn.metrics import accuracy_score, classification_report
import pandas as pd

print("Accuracy:", accuracy_score(y_test, y_pred))

print("\nClassification Report:")
report = classification_report(y_test, y_pred, zero_division=0, output_dict=True)
df_report = pd.DataFrame(report).transpose()
print(df_report)
```

```
Accuracy: 0.9523809523809523

Classification Report:
              precision    recall  f1-score    support
1              1.000000  1.000000  1.000000  12.000000
2              1.000000  1.000000  1.000000   2.000000
3              0.000000  0.000000  0.000000   1.000000
4              0.666667  1.000000  0.800000   2.000000
6              1.000000  1.000000  1.000000   3.000000
7              1.000000  1.000000  1.000000   1.000000
accuracy       0.952381  0.952381  0.952381   0.952381
macro avg      0.777778  0.833333  0.800000  21.000000
weighted avg   0.920635  0.952381  0.933333  21.000000
```

```
from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(7,5))
sns.heatmap(cm, annot=True, fmt='d')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```


Confusion Matrix
Actual
Predicted
0
1
2
3
4
5
12
0
0
0
0
0
0
2
0
0
0
0
0
0
1
0
0
0
0
0
2
0
0
0
0
0
0
3
0
0
0
0
0
0
0
1
12
10
8
6
4
2
0

Program 5

Use an appropriate data set for building the decision tree (ID3) and apply this knowledge to classify a new sample.

Screenshot

Lab 5

Q] KNN classifier model solve.

K = 3

Test data: X = (Age = 35, Salary = 100)

Dataset:

Person	Age	Salary	Target
A	18	50	N
B	23	55	N
C	24	70	N
D	41	60	Y
E	43	70	Y
F	38	40	Y

calculating euclidean distance,

$d = \sqrt{(age_1 - age_2)^2 + (salary_1 - salary_2)^2}$

distance from X(35, 100)

A(18,50) $d = \sqrt{(35-18)^2 + (100-50)^2} = \sqrt{2789} = 52.82$

B(23,55) $d = \sqrt{(35-23)^2 + (100-55)^2} = \sqrt{2169} = 46.57$

C(24,70) $d = \sqrt{(35-24)^2 + (100-70)^2} = \sqrt{1081} = 31.94$

D(41,60) $d = \sqrt{1636} = 40.45$

E(43,70) $d = \sqrt{964} = 31.05$

F(38,40) $d = \sqrt{3609} = 60.07$

sort distance,

E	31.05	Y
C	31.94	N
D	40.45	Y
B	46.57	N
A	52.82	N
F	60.07	Y

select K = 3 nearest neighbours

E	Y
C	N
D	Y

Y = 2

N = 1

majority class = Y

Target for X(35,100) = Y

P.T.O

For Iris dataset

Q] How to choose the K value? Demo w/ accuracy rate & error rate

The value of K represents the no. nearest neighbors used to classify a new data point

small K → overfit the data

large K → underfit the data

To choose the best K, we calculate accuracy rate & error rate for different K values, and select the K with highest accuracy & lowest error

Steps

1. Train KNN model for multiple K values
2. calculate accuracy score for each K
3. compute error rate = 1 - accuracy
4. select K

Q] What is the purpose of feature scaling? How to perform it? For Diabetes dataset

Feature Scaling is used to normalize the range of independent variables so that all features contribute equally to the model

This is needed because it calculates distance b/w data points. If one feature has a larger range than others, it will dominate the distance calculation

It ensures all features are on similar scale.

Feature Scaling can be performed by standardization.

$$X_{scaled} = \frac{X - Mean}{Std\ Deviation}$$

Results

For iris dataset: accuracy 1.0

confusion matrix $\begin{bmatrix} 10 & 0 & 0 \\ 0 & 9 & 0 \\ 0 & 0 & 11 \end{bmatrix}$

For diabetes dataset: accuracy : 0.6948051

confusion matrix : $\begin{bmatrix} 79 & 20 \\ 27 & 28 \end{bmatrix}$

For heart dataset: accuracy : 0.9180327

confusion matrix : $\begin{bmatrix} 27 & 2 \\ 3 & 29 \end{bmatrix}$

Code:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

data = pd.read_csv("iris (1).csv")
X = data.drop('species', axis=1)
y = data['species']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
y_pred = knn.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
Accuracy: 1.0
```

```
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

```
Confusion Matrix:
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
```

```
print("Classification Report:")
print(classification_report(y_test, y_pred))
```

```
Classification Report:
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       1.00      1.00      1.00         9
   virginica       1.00      1.00      1.00        11

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30
```

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

data = pd.read_csv("diabetes.csv")
```

```
X = data.drop('Outcome', axis=1)
y = data['Outcome']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
y_pred = knn.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
Accuracy: 0.6948051948051948
```

```
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

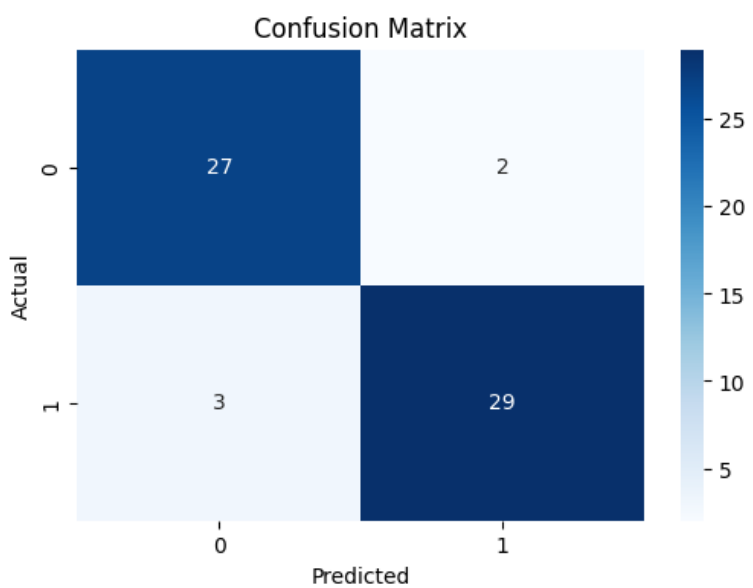
```
Confusion Matrix:
[[79 20]
 [27 28]]
```

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.preprocessing import StandardScaler

data = pd.read_csv("heart.csv")
X = data.drop('target', axis=1)
y = data['target']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

scores = []

for k in range(1,21):
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train, y_train)
    y_pred = knn.predict(X_test)
    scores.append(accuracy_score(y_test, y_pred))
```

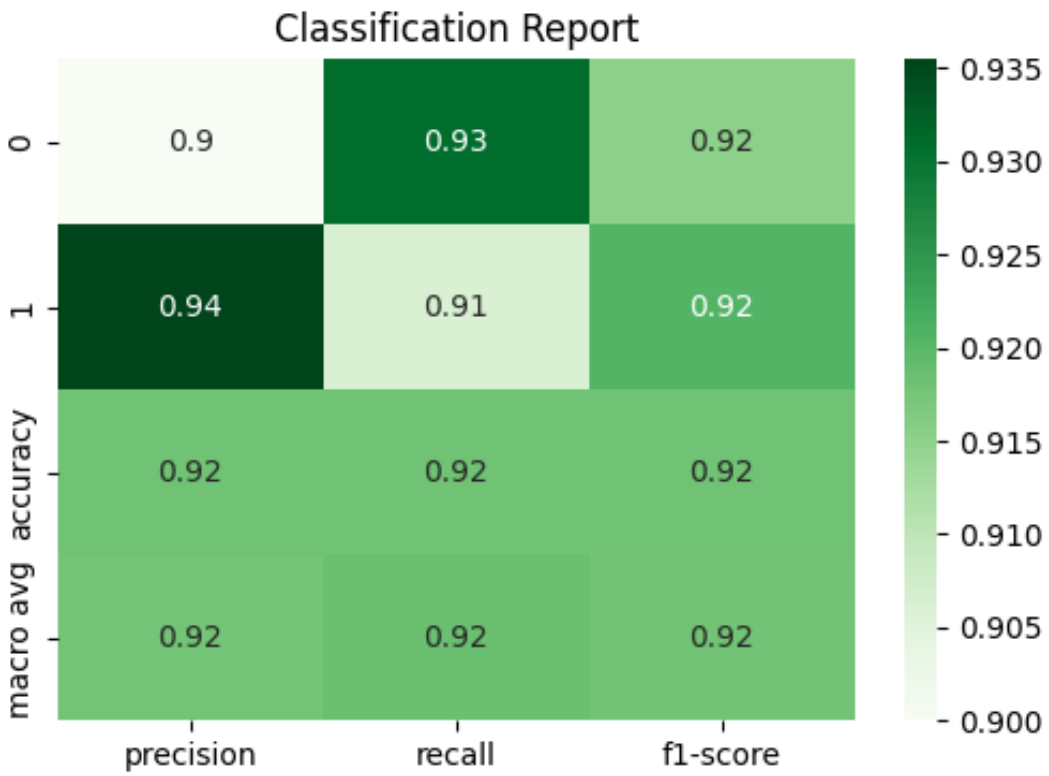
```
best_k = scores.index(max(scores)) + 1
print("Best K value:", best_k)

knn = KNeighborsClassifier(n_neighbors=best_k)
knn.fit(X_train, y_train)
y_pred = knn.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
Best K value: 7
Accuracy: 0.9180327868852459
```

```
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6,4))
sns.heatmap(cm, annot=True, cmap="Blues", fmt="d")
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```




```
report = classification_report(y_test, y_pred, output_dict=True)
report_df = pd.DataFrame(report).transpose()

plt.figure(figsize=(6,4))
sns.heatmap(report_df.iloc[:-1, :-1], annot=True, cmap="Greens")
plt.title("Classification Report")
plt.show()
```


Classification Report
0
1
accuracy
macro avg
0.9
0.93
0.92
0.94
0.91
0.92
0.92
0.92
0.92
0.92
0.92
0.92
precision
recall
f1-score
0.935
0.930
0.925
0.920
0.915
0.910
0.905
0.900

Program 6

Build KNN Classification model for a given dataset.

Screenshot

Date 13/03/2025
Page 18

Lab 6
Decision Tree

Q] Calculate entropy & info gain wrt to target variable "Classification"

Instance	a_2	a_3	Classification
1	Hot	High	No
2	Hot	High	No
6	Cool	High	No
7	Hot	High	No
8	Hot	Normal	Yes

Yes = 1 No = 4
$P(Yes) = 1/5 = 0.2$ $P(No) = 4/5 = 0.8$

$Entropy(S) = -(0.2 \log_2 0.2 + 0.8 \log_2 0.8) = 0.72$

For a_2 : a_2 = Hot
Yes = 1, No = 3
$P(Yes) = 1/4 = 0.25$ $P(No) = 3/4 = 0.75$

$Entropy(Hot) = -(0.25 \log_2 0.25 + 0.75 \log_2 0.75) = 0.81$

a_2 = Cool
Yes = 0, No = 1
Entropy (Cool) = 0

Weighted Entropy:

① $Entropy(S, a_2) = \frac{4}{5}(0.81) + \frac{1}{5}(0) = 0.648$

$IG(a_2) = Entropy(S) - Entropy(S, a_2) = 0.72 - 0.648 = 0.072$

For a_3 : a_3 = High
Yes = 0, No = 4
Entropy (High) = 0

a_3 = Normal
Yes = 1, No = 0
Entropy (Normal) = 0

Weighted Entropy:

② $Entropy(S, a_3) = \frac{4}{5}(0) + \frac{1}{5}(0) = 0$

$IG(a_3) = 0.72 - 0 = 0.72$

Since IG of a_3 is higher, the splitting node should be a_3

P.T.O

Date __/__/__
Page 20

For Iris dataset

Q] What was the accuracy score for the IRIS dataset?
The decision tree model achieved high accuracy of 1.0 on the IRIS dataset

Q] What does the confusion matrix tell you about the model's performance? Were there any misclassifications?
The confusion matrix shows that most samples are correctly classified into their respective class. There are very few misclassified mainly b/w versicolor and virginica because their features are similar
Overall, the model performs well on this dataset

$$\text{confusion matrix} = \begin{bmatrix} 10 & 0 & 0 \\ 0 & 9 & 0 \\ 0 & 0 & 11 \end{bmatrix}$$

For petrol consumption dataset

Q] Can you interpret the Regression Tree? What are the most imp features for predicting petrol consumption?
The regression tree splits the dataset based on imp features such as petrol tax, avg income, paved highways and population driver liscence to predict petrol consumption.
Features like avg income and petrol tax play a major role in predicting petrol consumption.

Q] How does the regression tree handle continuous target variables compared to the decision tree classifier?
Unlike decision tree classifiers that predict categories, the regression tree predicts continuous numeric values by splitting the data & calc the avg values in each leaf node

$$\text{Matrix} = \begin{bmatrix} 6 & 0 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 & 0 \\ 0 & 0 & 5 & 0 & 0 \\ 0 & 0 & 0 & 11 & 0 \\ 0 & 0 & 0 & 0 & 15 \end{bmatrix}$$

Results
MAE - 91.5
MSE - 17171.7
RMSE - 131.04833306836

Code:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

iris = pd.read_csv("iris (1).csv")
X = iris.drop("species", axis=1)
y = iris["species"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = DecisionTreeClassifier()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("IRIS Accuracy:", accuracy_score(y_test, y_pred))
print("IRIS Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

```
IRIS Accuracy: 1.0
IRIS Confusion Matrix:
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
```

```
from sklearn.preprocessing import LabelEncoder

drug = pd.read_csv("drug.csv")

le = LabelEncoder()
drug["Sex"] = le.fit_transform(drug["Sex"])
drug["BP"] = le.fit_transform(drug["BP"])
drug["Cholesterol"] = le.fit_transform(drug["Cholesterol"])
drug["Drug"] = le.fit_transform(drug["Drug"])

X = drug.drop("Drug", axis=1)
y = drug["Drug"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = DecisionTreeClassifier()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Drug Dataset Accuracy:", accuracy_score(y_test, y_pred))
print("Drug Dataset Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

```
Drug Dataset Accuracy: 1.0
Drug Dataset Confusion Matrix:
[[ 6  0  0  0  0]
 [ 0  3  0  0  0]
 [ 0  0  5  0  0]
 [ 0  0  0 11  0]
 [ 0  0  0  0 15]]
```

```
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error
import numpy as np

petrol = pd.read_csv("petrol_consumption.csv")

X = petrol.drop("Petrol_Consumption", axis=1)
y = petrol["Petrol_Consumption"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = DecisionTreeRegressor()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Mean Absolute Error:", mean_absolute_error(y_test, y_pred))
print("Mean Squared Error:", mean_squared_error(y_test, y_pred))
print("Root Mean Squared Error:", np.sqrt(mean_squared_error(y_test, y_pred)))
```

```
Mean Absolute Error: 91.5
Mean Squared Error: 17171.7
Root Mean Squared Error: 131.0408333306836
```

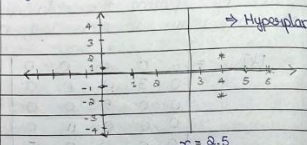
Program 7

Build Support vector machine model for a given dataset.

Screenshot

Lab 7
SVM

Q] Positive : (4,1), (4,-1), (6,0)
Negative : (1,0), (0,1), (0,-1)



⇒ Hyperplane

$x = 2.5$

Support vectors chosen ⇒ $S_1 = (1,0)$ −ve
⇒ $S_2 = (4,1)$ +ve
⇒ $S_3 = (4,-1)$ +ve

finding augmented SVs

$\tilde{S}_1 = \begin{pmatrix}1\\0\\1\end{pmatrix}, \quad \tilde{S}_2 = \begin{pmatrix}4\\1\\1\end{pmatrix}, \quad \tilde{S}_3 = \begin{pmatrix}4\\-1\\1\end{pmatrix}$

$$-1 = \alpha_1 \begin{pmatrix}1\\0\\1\end{pmatrix}\begin{pmatrix}1\\0\\1\end{pmatrix} + \alpha_2 \begin{pmatrix}4\\1\\1\end{pmatrix}\begin{pmatrix}1\\0\\1\end{pmatrix} + \alpha_3 \begin{pmatrix}4\\-1\\1\end{pmatrix}\begin{pmatrix}1\\0\\1\end{pmatrix}$$

$$1 = \alpha_1 \begin{pmatrix}1\\0\\1\end{pmatrix}\begin{pmatrix}4\\1\\1\end{pmatrix} + \alpha_2 \begin{pmatrix}4\\1\\1\end{pmatrix}\begin{pmatrix}4\\1\\1\end{pmatrix} + \alpha_3 \begin{pmatrix}4\\-1\\1\end{pmatrix}\begin{pmatrix}4\\1\\1\end{pmatrix}$$

$$1 = \alpha_1 \begin{pmatrix}1\\0\\1\end{pmatrix}\begin{pmatrix}4\\-1\\1\end{pmatrix} + \alpha_2 \begin{pmatrix}4\\1\\1\end{pmatrix}\begin{pmatrix}4\\-1\\1\end{pmatrix} + \alpha_3 \begin{pmatrix}4\\-1\\1\end{pmatrix}\begin{pmatrix}4\\-1\\1\end{pmatrix}$$

$2\alpha_1 + 5\alpha_2 + 5\alpha_3 = -1$
$5\alpha_1 + 18\alpha_2 + 16\alpha_3 = 1$
$5\alpha_1 + 16\alpha_2 + 18\alpha_3 = 1$

$\Rightarrow \alpha_1 = -2.44$
$\alpha_2 = 0.388$
$\alpha_3 = 0.388$

$$-2.44\begin{pmatrix}1\\0\\1\end{pmatrix} + 0.388\begin{pmatrix}4\\1\\1\end{pmatrix} + 0.388\begin{pmatrix}4\\-1\\1\end{pmatrix} = \tilde{w}$$

$$\begin{pmatrix}0.664\\0\\-1.664\end{pmatrix} = \tilde{w}$$

$b = -1.664, \quad w = \begin{pmatrix}0.664\\0\end{pmatrix}$

$wx + b = 0$
$0.664x - 1.664 = 0$
$x = 2.5$

1. For Iris dataset.
Q] What is the accuracy score of the classifier using the linear kernel and RBF kernel?
Accuracy Scores :
i) Linear Kernel Accuracy ~ 0.96-1.00
ii) RBF Kernel Accuracy ~ 0.97-1.00

Q] Which kernel gave better performance on the IRIS dataset? Why do you think that is the case?
* RBF kernel performed slightly better compared to linear kernel
* The iris dataset is almost linearly separable, so even the linear kernel performs well
* However, RBF can handle non-linear boundaries, so it may capture small complex patterns better
* Hence RBF gives equal or slightly higher accuracy.

2. For Letter Recognition Dataset
Q] Present & interpret the confusion matrix?
The confusion matrix is a 26 × 26 matrix
Diagonal elements → correct classif
off diagonal elements → misclassificat
Most values are conc. along the diagonal → good accuracy
Some letters are frequently confused
O & Q, I & L, C & G, U & V

Q] AUC score and how does it reflect the model performance?
AUC Score ~ 0.99 - 1.00
AUC close to 1 indicates :
1. Excellent classification performance
2. High True +ve Rate
3. Low False +ve Rate

Q] Compare SVM performance w/ both datasets.

Feature	Iris Dataset	Letter Dataset
no of class	3	26
complexity	low	high
accuracy	very high	slightly lower
model	excellent	good but more challenging

Code:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix

data = pd.read_csv("iris.csv")
X = data.iloc[:, :-1]
y = data.iloc[:, -1]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

linear_model = SVC(kernel='linear')
linear_model.fit(X_train, y_train)
y_pred_linear = linear_model.predict(X_test)

print(" Linear Kernel ")
print("Accuracy:", accuracy_score(y_test, y_pred_linear))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_linear))
```

```
 Linear Kernel
Accuracy: 1.0
Confusion Matrix:
 [[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
```

```
rbf_model = SVC(kernel='rbf')
rbf_model.fit(X_train, y_train)
y_pred_rbf = rbf_model.predict(X_test)

print(" RBF Kernel ")
print("Accuracy:", accuracy_score(y_test, y_pred_rbf))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_rbf))
```

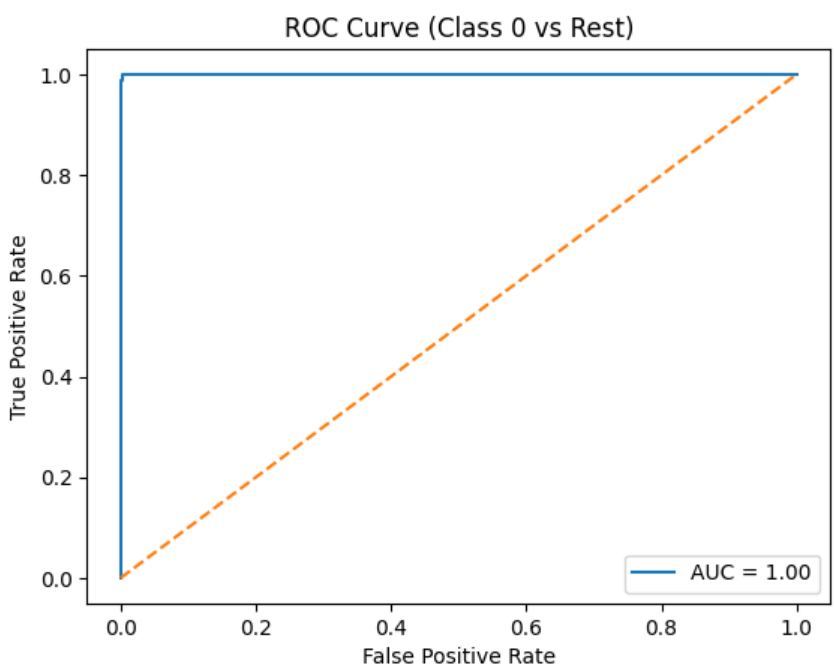
```
 RBF Kernel
Accuracy: 1.0
Confusion Matrix:
 [[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
```

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix, roc_curve, auc
from sklearn.preprocessing import label_binarize
import matplotlib.pyplot as plt
```

```
data = pd.read_csv("letter-recognition.csv")

X = data.iloc[:, 1:]
y = data.iloc[:, 0]

classes = sorted(y.unique())
y_bin = label_binarize(y, classes=classes)
X_train, X_test, y_train, y_test, y_train_bin, y_test_bin = train_test_split(
    X, y, y_bin, test_size=0.2, random_state=42)

model = SVC(kernel='rbf', probability=True)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))

y_score = model.predict_proba(X_test)

fpr, tpr, _ = roc_curve(y_test_bin[:, 0], y_score[:, 0])
roc_auc = auc(fpr, tpr)

plt.figure()
plt.plot(fpr, tpr, label="AUC = %0.2f" % roc_auc)
plt.plot([0, 1], [0, 1], linestyle='--')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve (Class 0 vs Rest)")
plt.legend()
plt.show()

print("AUC Score:", roc_auc)
```

```
AUC Score: 0.9999424885717821
```

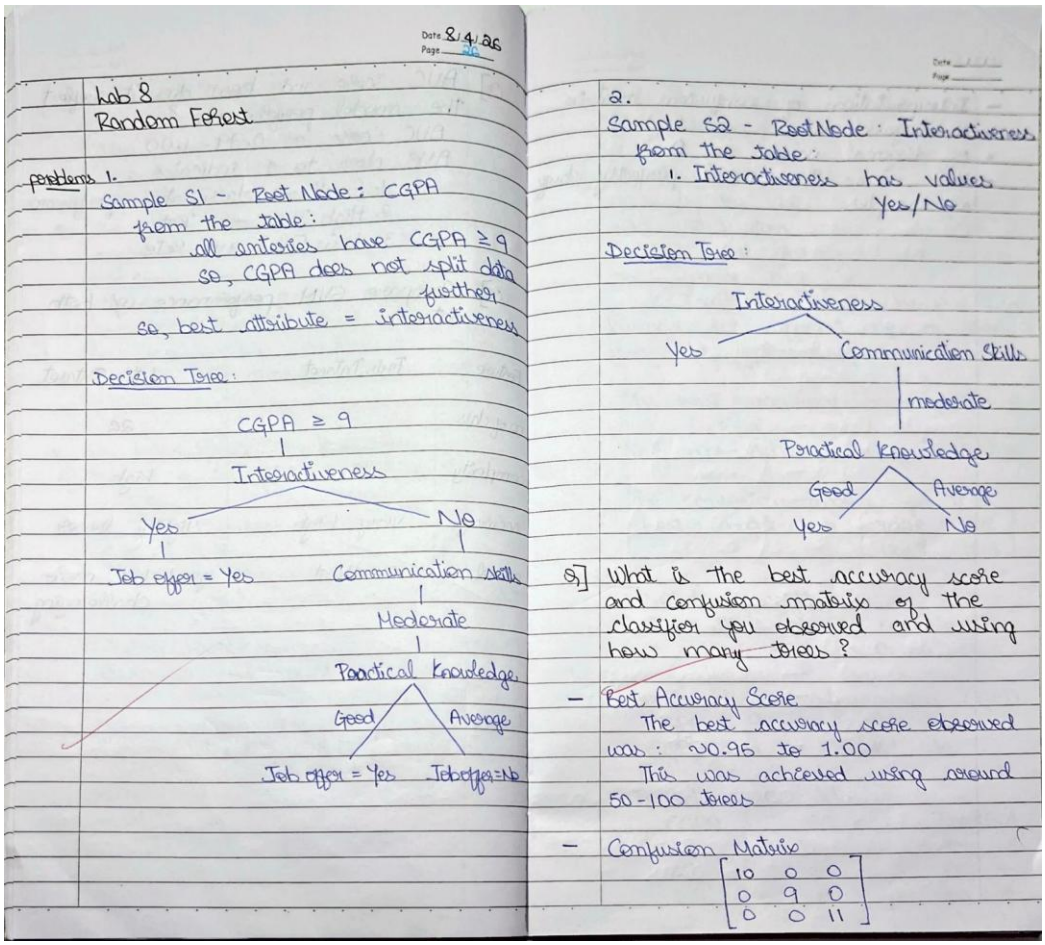
```
Accuracy: 0.9385
Confusion Matrix:
[[144   0   1   0   0   0   1   0   0   0   1   0   1   0   0   0   0   0
    0   0   1   0   0   0   0   0]
 [  0 143   0   5   0   1   0   0   0   0   0   0   0   0   0   0   0   4
    0   0   0   0   0   0   0   0]
 [  0   0 123   0   2   0   3   1   0   0   1   0   0   0   4   0   0   2
    0   0   1   0   0   0   0   0]
 [  0   1   0 153   0   0   0   2   0   0   0   0   0   0   0   0   0   0
    0   0   0   0   0   0   0   0]
 [  0   3   1   0 138   0   5   0   0   0   0   0   0   0   0   0   1   0
    0   0   0   0   0   0   0   1]
 [  0   2   0   0   1 134   0   0   1   0   0   0   0   0   0   0   0   0
    1   1   0   0   0   0   0   0]
 [  1   0   1   4   0   0 149   0   0   0   2   0   0   0   0   0   0   2
    0   0   0   1   0   0   0   0]
 [  0   4   0   3   0   0   0 106   0   0   5   0   0   1   2   1   1  13
    0   0   2   0   0   0   1   0]
 [  0   0   0   1   0   2   0   0 134   7   0   0   0   0   0   0   0   0
    0   0   0   0   0   2   0   0]
 [  0   0   0   1   1   1   0   0   3 139   0   0   0   0   1   0   0   0
    3   0   0   0   0   0   0   0]
 [  0   0   0   2   0   0   0   1   0   0 112   0   0   0   0   0   0  12
    0   0   0   0   0   3   0   0]
 [  0   0   1   0   4   0   3   0   0   0   0 142   0   0   1   0   1   1
    2   0   0   0   0   0   0   0]
 [  0   2   0   0   0   0   0   0   0   0   0   0 164   0   0   0   0   2
    0   0   0   0   0   0   0   0]
 [  1   0   0   3   0   0   0   0   0   0   0   0   0 139   5   0   0   2
    0   0   0   0   1   0   0   0]
 [  0   0   0   4   0   0   0   0   0   0   0   0   1   0 134   0   2   0
    0   0   1   0   3   0   0   0]
 [  0   1   0   0   0  14   8   0   0   0   0   0   0   0   0 148   0   0
    0   0   0   0   0   0   2   0]
 [  0   3   0   1   3   0   0   0   0   0   0   0   0   0   0   0 159   0
    0   0   0   0   0   0   0   0]
 [  0   5   0   2   0   0   0   0   0   0   2   1   0   2   0   0   0 148
    0   0   0   0   0   0   0   0]
 [  0   2   0   0   1   0   0   0   0   0   0   0   0   0   0   0   0   0
  167   0   0   0   0   0   0   1]
 [  0   1   0   0   1   0   0   0   0   0   2   0   0   0   0   0   0   1
    0 153   1   0   0   2   1   1]
 [  0   0   0   0   0   0   0   0   0   0   2   0   4   0   2   0   0   0
    0   0 170   2   3   0   0   0]
 [  1   5   0   0   0   1   0   0   0   0   0   0   1   0   0   0   0   1
    0   0   0 147   2   0   0   0]
 [  0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
    0   0   3   0 145   0   0   0]
 [  0   1   0   2   0   0   0   0   0   0   1   0   0   0   0   0   0   0
    0   0   0   0   0 150   0   0]
 [  1   0   0   1   0   0   0   0   0   0   0   0   0   0   0   0   0   0
    0   1   0   1   0   0 164   0]
 [  1   0   0   0   0   0   0   0   0   1   0   0   0   0   0   0   0   1
    4   0   0   0   0   0   0 125]]
```



Program 8

Implement Random forest ensemble method on a given dataset.

Screenshot



Date 8/4/26
Page No

Lab 8
Random Forest

problems 1.

Sample S1 - Root Node : CGPA
from the table :
all entries have CGPA ≥ 9
so, CGPA does not split data further
so, best attribute = interactiveness

Decision Tree :

CGPA ≥ 9
Interactiveness
Yes → Job offer = Yes
No → Communication skills
Moderate
Practical Knowledge
Good → Job offer = Yes
Average → Job offer = No

2.

Sample S2 - Root Node : Interactiveness
from the table :
1. Interactiveness has values Yes/No

Decision Tree :

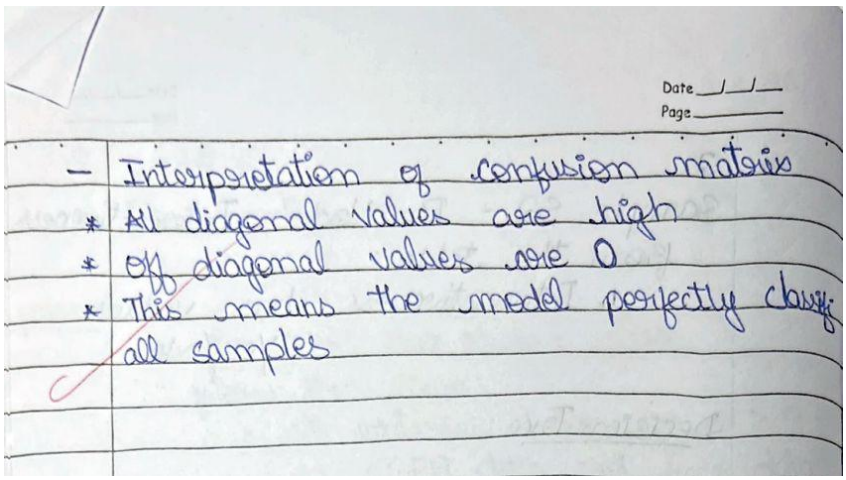
Interactiveness
Yes
Communication Skills
moderate
Practical Knowledge
Good → Yes
Average → No

Q] What is the best accuracy score and confusion matrix of the classifier you observed and using how many trees ?

- Best Accuracy Score
The best accuracy score observed was : ~0.95 to 1.00
This was achieved using around 50 - 100 trees

- Confusion Matrix

$$\begin{bmatrix} 10 & 0 & 0 \\ 0 & 9 & 0 \\ 0 & 0 & 11 \end{bmatrix}$$



Date / /
Page

- Interpretation of confusion matrix
* All diagonal values are high
* Off diagonal values are 0
* This means the model perfectly classifi all samples

Code:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

data = pd.read_csv("iris.csv")

X = data.drop("species", axis=1)
y = data["species"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)

rf_default = RandomForestClassifier(n_estimators=10, random_state=42)
rf_default.fit(X_train, y_train)

y_pred_default = rf_default.predict(X_test)

accuracy_default = accuracy_score(y_test, y_pred_default)
print("Accuracy with 10 trees:", accuracy_default)
```

```
•••  Accuracy with 10 trees: 1.0
```

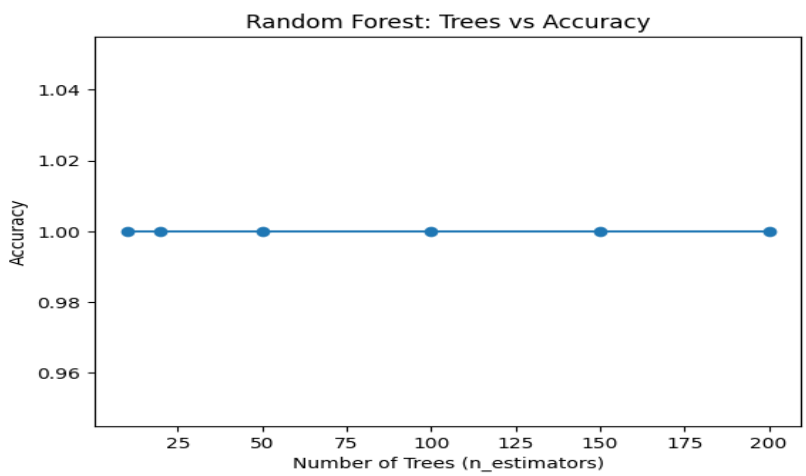
```
best_score = 0
best_n = 0

for n in [10, 20, 50, 100, 150, 200]:
    rf = RandomForestClassifier(n_estimators=n, random_state=42)
    rf.fit(X_train, y_train)

    y_pred = rf.predict(X_test)
    acc = accuracy_score(y_test, y_pred)

    print(f"Trees: {n}, Accuracy: {acc}")

    if acc > best_score:
        best_score = acc
        best_n = n
```

```
•••  Trees: 10, Accuracy: 1.0
     Trees: 20, Accuracy: 1.0
     Trees: 50, Accuracy: 1.0
     Trees: 100, Accuracy: 1.0
     Trees: 150, Accuracy: 1.0
     Trees: 200, Accuracy: 1.0
```

```
print("\nBest Accuracy:", best_score)
print("Best Number of Trees:", best_n)
```

```
...
    Best Accuracy: 1.0
    Best Number of Trees: 10
```

```
import matplotlib.pyplot as plt

trees = [10, 20, 50, 100, 150, 200]
accuracies = []

for n in trees:
    rf = RandomForestClassifier(n_estimators=n, random_state=42)
    rf.fit(X_train, y_train)

    y_pred = rf.predict(X_test)
    acc = accuracy_score(y_test, y_pred)

    accuracies.append(acc)

plt.figure()
plt.plot(trees, accuracies, marker='o')
plt.xlabel("Number of Trees (n_estimators)")
plt.ylabel("Accuracy")
plt.title("Random Forest: Trees vs Accuracy")
plt.show()
```



Program 9

Implement Boosting ensemble method on a given dataset.

Screenshot

Date 8/4/26
Page 29

Lab 9
ADA Boost

Q] For income.csv dataset
What is the best accuracy score and confusion matrix of the classifier you observed and using how many trees?

The ADA Boost classifier was trained w/ different no of trees to find the optimal performance.

The best accuracy score obtained is 0.888
This was achieved using 200 trees

The confusion matrix for the best model = $\begin{bmatrix} 7038 & 386 \\ 1343 & 1112 \end{bmatrix}$

Problem.

Initialize weights :
Since there are 6 samples :

$$w_i = \frac{1}{6} = 0.1667$$

Create decision stump on CGPA
CGPA ≥ 9 → Predict Yes
CGPA < 9 → Predict No

Prediction vs Actual

CGPA	Actual	Predicted
≥9	Yes	Yes ✓
<9	Yes	No ✗
≥9	No	Yes ✗
<9	No	No ✓
≥9	Yes	Yes ✓
≥9	Yes	Yes ✓

Calculate Error
Misclassified points :
- Row 2
- Row 3

$$\text{Error} = w_2 + w_3 = 0.1667 + 0.1667 = 0.3334$$

Compute Alpha

$$\alpha = \frac{1}{2} \ln\left(\frac{1 - \text{error}}{\text{error}}\right)$$

$$\alpha = \frac{1}{2} \ln\left(\frac{1 - 0.3334}{0.3334}\right)$$

$$\alpha \approx 0.3466$$

Update weights
correct → decrease weight
misclassified → increase weight

$$w_i^{(new)} = w_i \cdot e^{\pm\alpha}$$

misclassified : $0.1667 \times e^{0.3466} = 0.235$

correctly classified : $0.1667 \times e^{-0.3466} = 0.118$

Normalized weights
Total ≈ 1, so normalized weights
- misclassified → higher weight
- correct → lower weight

Decision stump based on CGPA
if CGPA ≥ 9 - Predict Yes
if CGPA < 9 - Predict No

Error = 0.3334
α = 0.3466

Code:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import AdaBoostClassifier
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt

df = pd.read_csv("income.csv")
print(df.head())

df = df.dropna()

le = LabelEncoder()
for col in df.columns:
    if df[col].dtype == 'object':
        df[col] = le.fit_transform(df[col])

X = df.iloc[:, :-1]
y = df.iloc[:, -1]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

```
...     age  fnlwgt  education_num  capital_gain  capital_loss  hours_per_week
    0   39   77516             13          2174             0              40
    1   50   83311             13             0             0              13
    2   38  215646              9             0             0              40
    3   53  234721              7             0             0              40
    4   28  338409             13             0             0              40

       income_level
    0             0
    1             0
    2             0
    3             0
    4             0
```

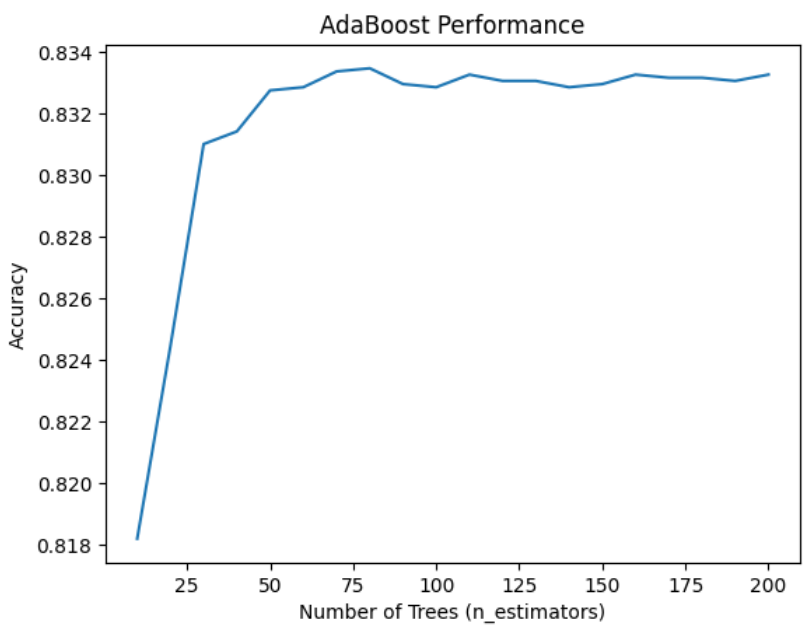
```
model = AdaBoostClassifier(n_estimators=10, random_state=42)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy with n_estimators = 10:", accuracy)

estimators_range = range(10, 201, 10)
scores = []
```

```
for n in estimators_range:
    model = AdaBoostClassifier(n_estimators=n, random_state=42)
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    score = accuracy_score(y_test, y_pred)
    scores.append(score)
```

```
Accuracy with n_estimators = 10: 0.8182004299314157
```

```
best_score = max(scores)
best_n = list(estimators_range)[scores.index(best_score)]

print("Best Accuracy:", best_score)
print("Best n_estimators:", best_n)

plt.plot(estimators_range, scores)
plt.xlabel("Number of Trees (n_estimators)")
plt.ylabel("Accuracy")
plt.title("AdaBoost Performance")
plt.show()
```

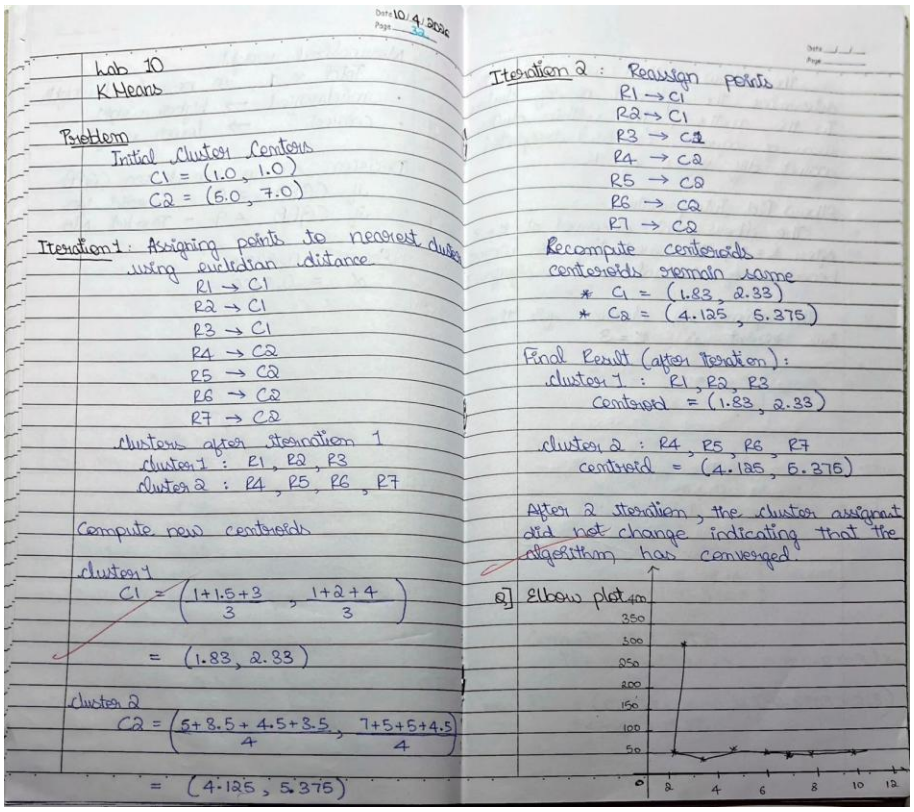
```
Best Accuracy: 0.8334527587265841
Best n_estimators: 80
```



Program 10

Build k-Means algorithm to cluster a set of data stored in a .CSV file.

Screenshot



Lab 10
K Means

Problem
Initial Cluster Centers
C1 = (1.0, 1.0)
C2 = (5.0, 7.0)

Iteration 1: Assigning points to nearest cluster using euclidian distance
R1 → C1
R2 → C1
R3 → C1
R4 → C2
R5 → C2
R6 → C2
R7 → C2

clusters after iteration 1
cluster 1 : R1, R2, R3
cluster 2 : R4, R5, R6, R7

Compute new centroids

cluster 1
$C1 = \left(\frac{1+1.5+3}{3}, \frac{1+2+4}{3}\right)$
$= (1.83, 2.33)$

cluster 2
$C2 = \left(\frac{5+8.5+4.5+3.5}{4}, \frac{7+5+5+4.5}{4}\right)$
$= (4.125, 5.375)$

Iteration 2 : Reassign points
R1 → C1
R2 → C1
R3 → C1
R4 → C2
R5 → C2
R6 → C2
R7 → C2

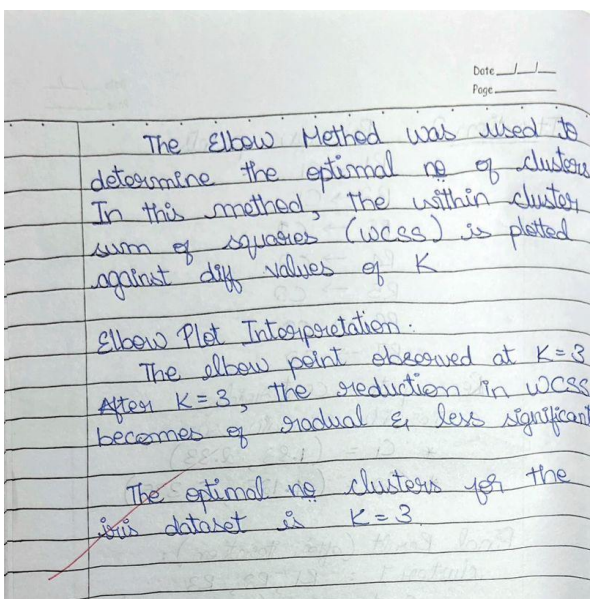
Recompute centroids
centroids remain same
* C1 = (1.83, 2.33)
* C2 = (4.125, 5.375)

Final Result (after iteration):
cluster 1 : R1, R2, R3
Centroid = (1.83, 2.33)

cluster 2 : R4, R5, R6, R7
centroid = (4.125, 5.375)

After 2 iteration, the cluster assignment did not change indicating that the algorithm has converged.

Q] Elbow plot



The Elbow Method was used to determine the optimal no of clusters. In this method, the within cluster sum of squares (WCSS) is plotted against diff values of K

Elbow Plot Interpretation:
The elbow point observed at K=3. After K=3, the reduction in WCSS becomes gradual & less significant.

The optimal no clusters for the iris dataset is K=3.

Code:

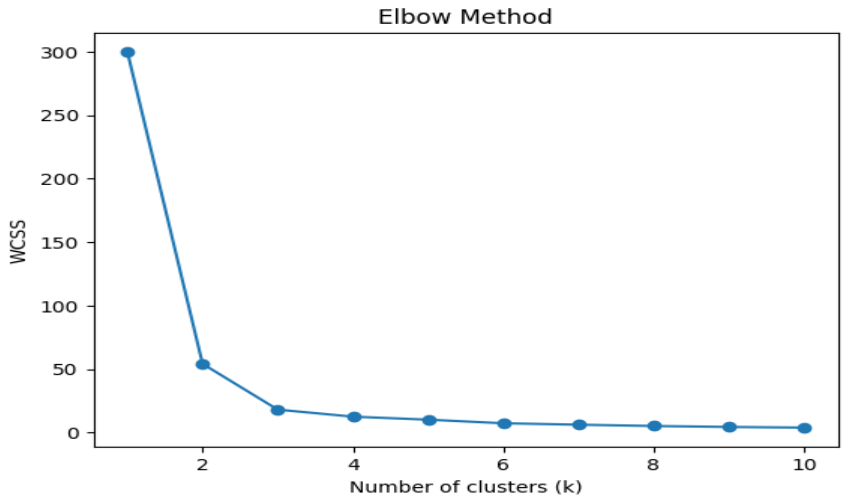
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

df = pd.read_csv('iris.csv')

print(df.head())
```

```
...     sepal_length  sepal_width  petal_length  petal_width species
    0           5.1          3.5           1.4          0.2  setosa
    1           4.9          3.0           1.4          0.2  setosa
    2           4.7          3.2           1.3          0.2  setosa
    3           4.6          3.1           1.5          0.2  setosa
    4           5.0          3.6           1.4          0.2  setosa
```

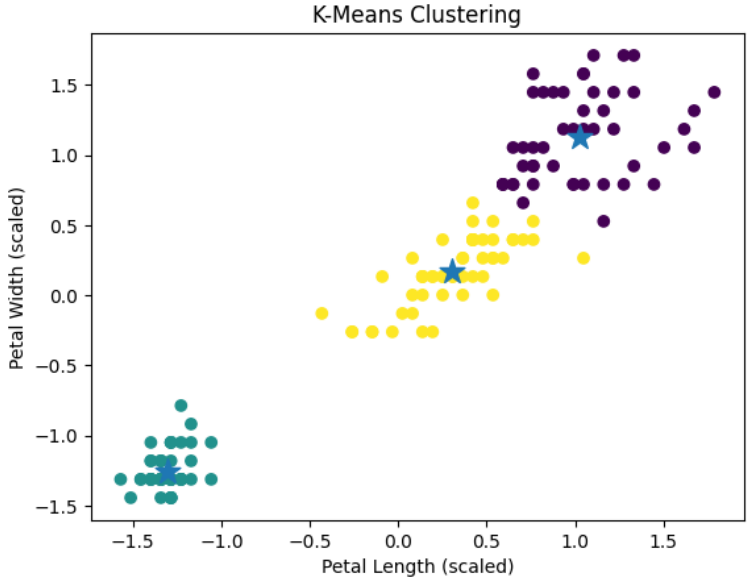
```
X = df[['petal_length', 'petal_width']]
print(X.head())
```

```
...     petal_length  petal_width
    0           1.4          0.2
    1           1.4          0.2
    2           1.3          0.2
    3           1.5          0.2
    4           1.4          0.2
```

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

wcss = []

for k in range(1, 11):
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(X_scaled)
    wcss.append(kmeans.inertia_)

plt.plot(range(1, 11), wcss, marker='o')
plt.title('Elbow Method')
plt.xlabel('Number of clusters (k)')
plt.ylabel('WCSS')
plt.show()
```



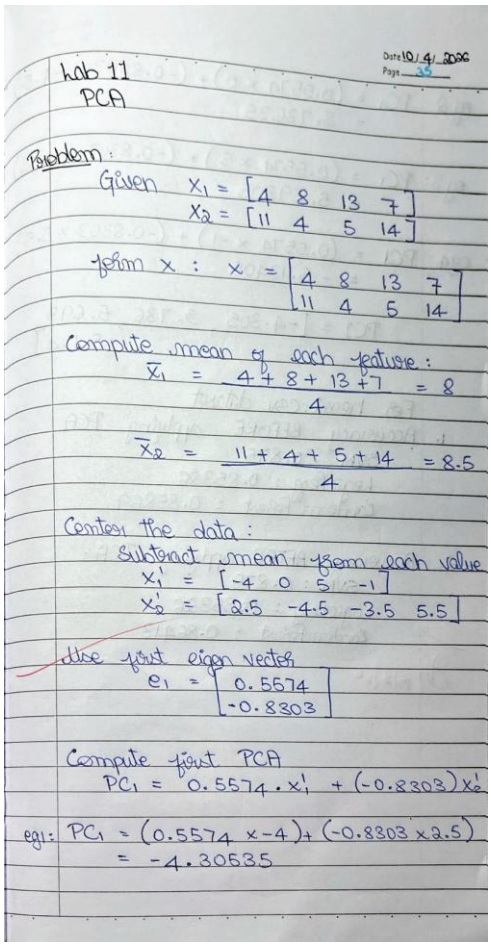

```
kmeans = KMeans(n_clusters=3, random_state=42)
y_pred = kmeans.fit_predict(X_scaled)

df['Cluster'] = y_pred

plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=y_pred)
plt.xlabel('Petal Length (scaled)')
plt.ylabel('Petal Width (scaled)')
plt.title('K-Means Clustering')

plt.scatter(kmeans.cluster_centers_[:, 0],
            kmeans.cluster_centers_[:, 1],
            s=200, marker='*')

plt.show()
```



Program 11

Implement Dimensionality reduction using Principal Component Analysis (PCA) method.

Screenshot



lab 11
PCA

Problem:

Given $x_1 = [4 \quad 8 \quad 13 \quad 7]$
$x_2 = [11 \quad 4 \quad 5 \quad 14]$

form x : $x = \begin{bmatrix} 4 & 8 & 13 & 7 \\ 11 & 4 & 5 & 14 \end{bmatrix}$

Compute mean of each feature:

$\bar{x}_1 = \frac{4+8+13+7}{4} = 8$

$\bar{x}_2 = \frac{11+4+5+14}{4} = 8.5$

Center the data:
Subtract mean from each value

$x_1' = [-4 \quad 0 \quad 5 \quad -1]$
$x_2' = [2.5 \quad -4.5 \quad -3.5 \quad 5.5]$

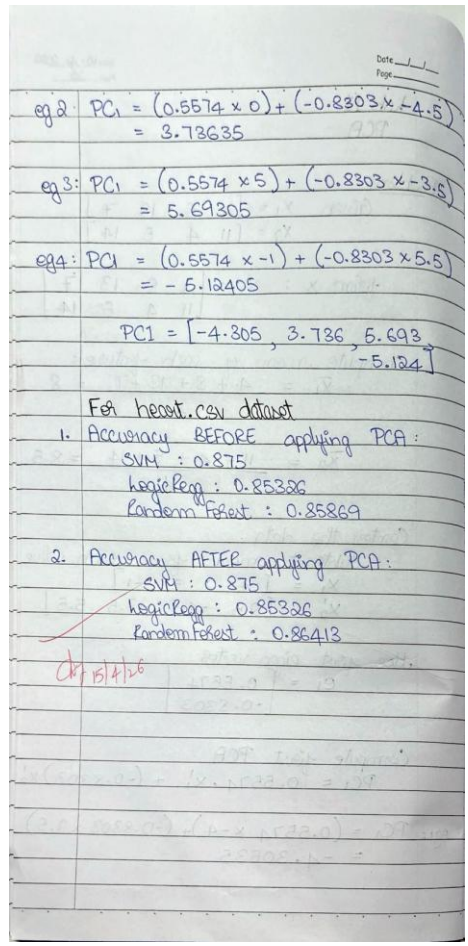
Use first eigen vector

$e_1 = \begin{bmatrix} 0.5574 \\ -0.8303 \end{bmatrix}$

Compute first PCA

$PC_1 = 0.5574 \cdot x_1' + (-0.8303) x_2'$

eg1: $PC_1 = (0.5574 \times -4) + (-0.8303 \times 2.5)$
$= -4.30535$



eg2: $PC_1 = (0.5574 \times 0) + (-0.8303 \times -4.5)$
$= 3.73635$

eg3: $PC_1 = (0.5574 \times 5) + (-0.8303 \times -3.5)$
$= 5.69305$

eg4: $PC_1 = (0.5574 \times -1) + (-0.8303 \times 5.5)$
$= -5.12405$

$PC1 = [-4.305, 3.736, 5.693, -5.124]$

For heart.csv dataset

1. Accuracy BEFORE applying PCA:
   SVM : 0.875
   logicReg : 0.85326
   Random Forest : 0.85869

2. Accuracy AFTER applying PCA:
   SVM : 0.875
   logicReg : 0.85326
   Random Forest : 0.86413

Ch 15/4/26

Code:

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, OneHotEncoder, StandardScaler
from sklearn.compose import ColumnTransformer

from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier

from sklearn.decomposition import PCA
from sklearn.metrics import accuracy_score
```

```
df = pd.read_csv('heart.csv')
print(df.head())
```

```
   Age Sex ChestPainType  RestingBP  Cholesterol  FastingBS RestingECG  MaxHR  \
0   40   M           ATA        140          289          0     Normal    172
1   49   F           NAP        160          180          0     Normal    156
2   37   M           ATA        130          283          0         ST     98
3   48   F           ASY        138          214          0     Normal    108
4   54   M           NAP        150          195          0     Normal    122

   ExerciseAngina  Oldpeak ST_Slope  HeartDisease
0               N      0.0       Up             0
1               N      1.0     Flat             1
2               N      0.0       Up             0
3               Y      1.5     Flat             1
4               N      0.0       Up             0
```

```
X = df.drop('HeartDisease', axis=1)
y = df['HeartDisease']

X_df = df.drop('HeartDisease', axis=1)

cat_cols = X_df.select_dtypes(include=['object']).columns
num_cols = X_df.select_dtypes(exclude=['object']).columns

preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), num_cols),
        ('cat', OneHotEncoder(handle_unknown='ignore', drop='first'), cat_cols)],
    remainder='passthrough')

X = preprocessor.fit_transform(X_df)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

svm = SVC()
lr = LogisticRegression(max_iter=1000)
rf = RandomForestClassifier()

svm.fit(X_train, y_train)
lr.fit(X_train, y_train)
rf.fit(X_train, y_train)

svm_pred = svm.predict(X_test)
lr_pred = lr.predict(X_test)
rf_pred = rf.predict(X_test)

print("Accuracy WITHOUT PCA:")
print("SVM:", accuracy_score(y_test, svm_pred))
print("Logistic Regression:", accuracy_score(y_test, lr_pred))
```

```
print("Random Forest:", accuracy_score(y_test, rf_pred))
```

```
Accuracy WITHOUT PCA:
SVM: 0.8586956521739131
Logistic Regression: 0.8532608695652174
Random Forest: 0.8641304347826086
```

```
pca = PCA(n_components=0.95)
X_pca = pca.fit_transform(X)
X_train_pca, X_test_pca, y_train, y_test = train_test_split(
    X_pca, y, test_size=0.2, random_state=42)

svm.fit(X_train_pca, y_train)
lr.fit(X_train_pca, y_train)
rf.fit(X_train_pca, y_train)

svm_pred_pca = svm.predict(X_test_pca)
lr_pred_pca = lr.predict(X_test_pca)
rf_pred_pca = rf.predict(X_test_pca)

print("\nAccuracy WITH PCA:")
print("SVM:", accuracy_score(y_test, svm_pred_pca))
print("Logistic Regression:", accuracy_score(y_test, lr_pred_pca))
print("Random Forest:", accuracy_score(y_test, rf_pred_pca))
```

```
Accuracy WITH PCA:
SVM: 0.842391304347826
Logistic Regression: 0.8260869565217391
Random Forest: 0.8369565217391305
```